Pesquisa em Memória Primária

- Como recuperar informação a partir de uma grande massa de informação previamente armazenada?
- A informação é dividida em registros:
  - Cada registro possui uma chave de pesquisa.
- O objetivo:
  - encontrar ocorrências de registros com chaves iguais à chave da pesquisa.
  - A pesquisa pode ser com sucesso ou sem sucesso

Pesquisa em Memória Primária

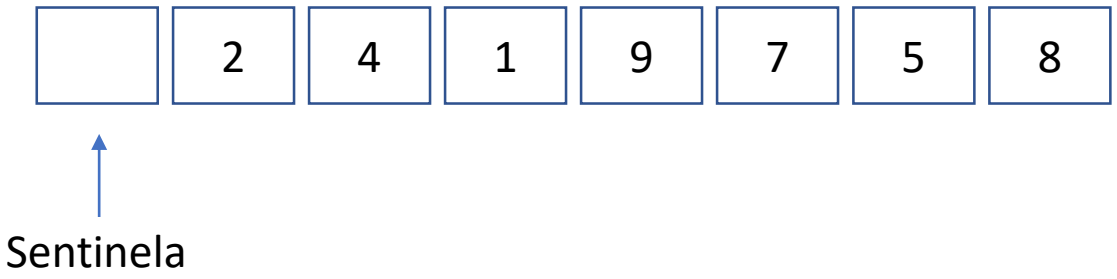
- Há diversos métodos de pesquisa;
- A escolha do método mais apropriado, depende:
  - Quantidade dos dados envolvidos;
  - Arquivo estar sujeito a inserções e retiradas frequentes (o objetivo é minimizar o tempo de pesquisa, sem preocupação com o tempo de estruturar o arquivo).

Pesquisa Sequencial

- Método de pesquisa mais simples:
  - a partir do primeiro registro, pesquise sequencialmente até encontrar a chave procurada; então pare.
- Armazenamento de um conjunto de registros por meio do tipo estruturado arranjo

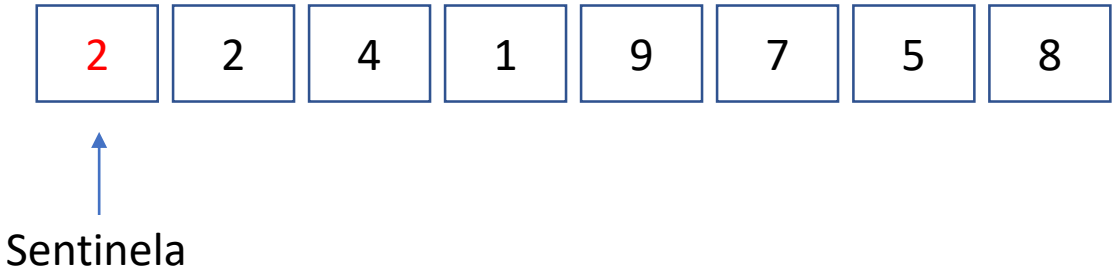
Pesquisa Sequencial

Chave pesquisa: reg = 2 – pior caso



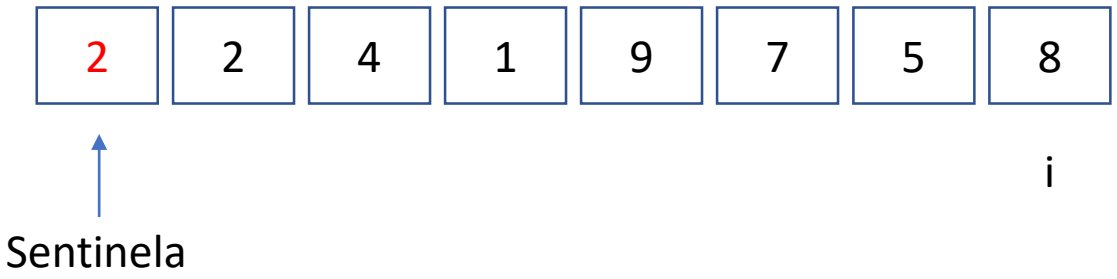
Pesquisa Sequencial

Chave pesquisa: reg = 2 – pior caso



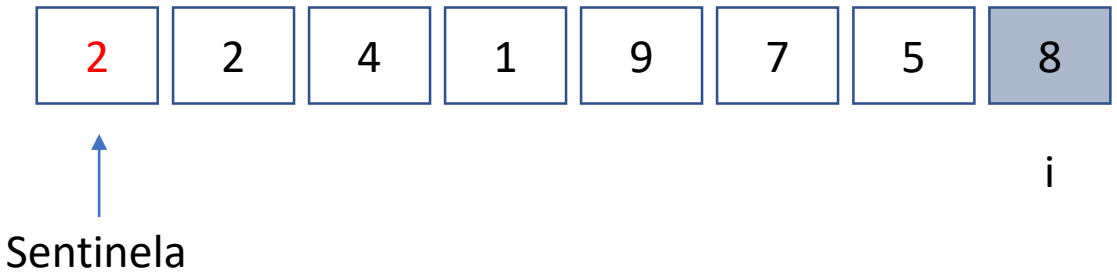
Pesquisa Sequencial

Chave pesquisa: reg = 2 – pior caso



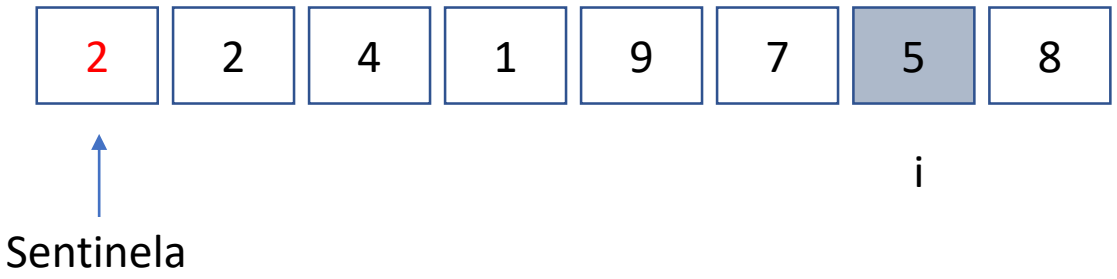
Pesquisa Sequencial

Chave pesquisa: reg = 2 – pior caso



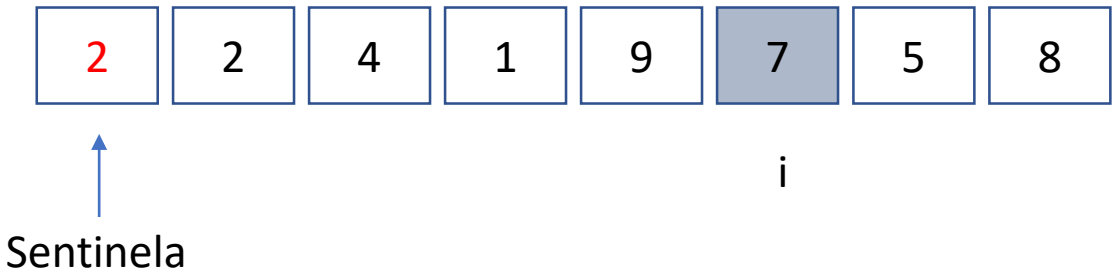
Pesquisa Sequencial

Chave pesquisa: reg = 2 – pior caso



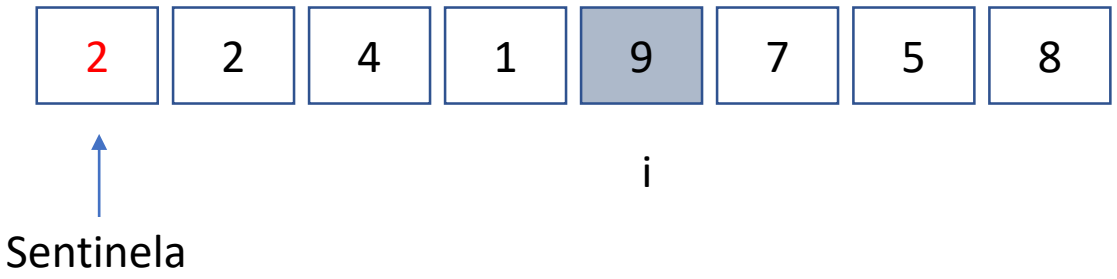
Pesquisa Sequencial

Chave pesquisa: reg = 2 – pior caso



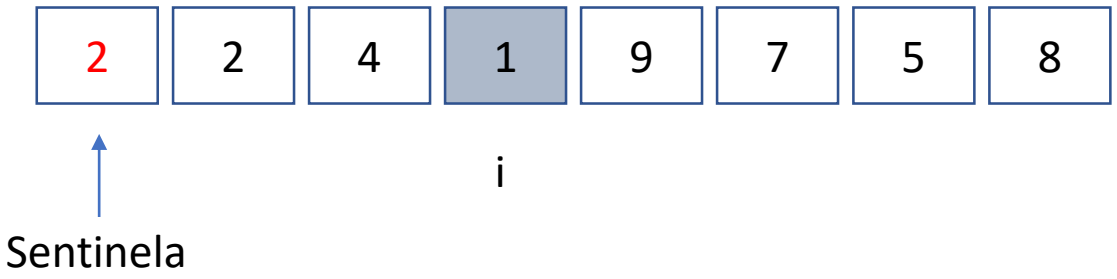
Pesquisa Sequencial

Chave pesquisa: reg = 2 – pior caso



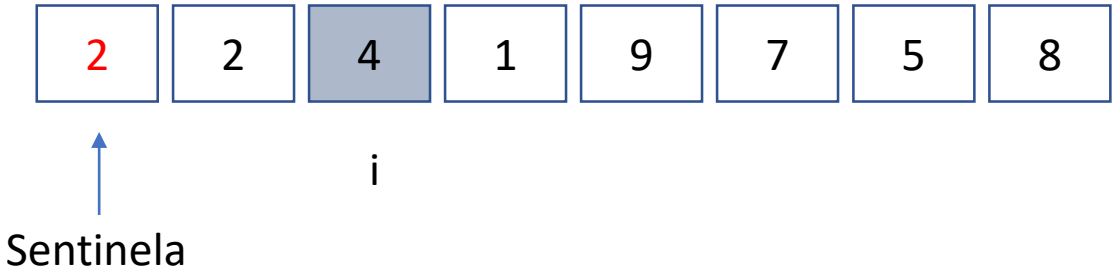
Pesquisa Sequencial

Chave pesquisa: reg = 2 – pior caso



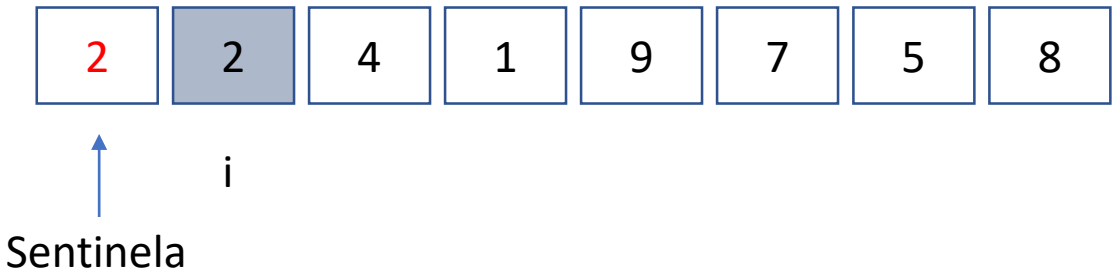
Pesquisa Sequencial

Chave pesquisa: reg = 2 – pior caso



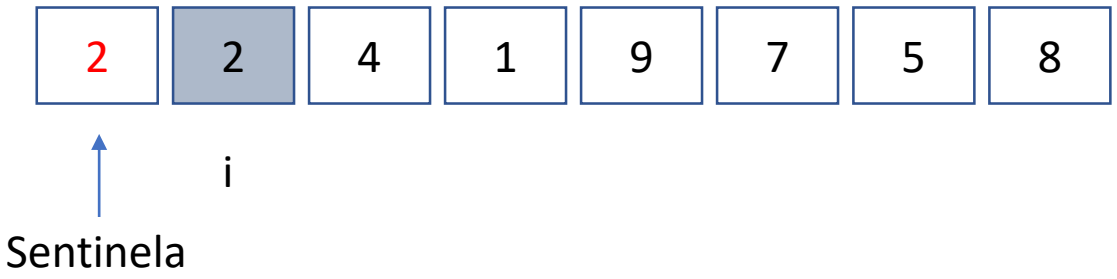
Pesquisa Sequencial

Chave pesquisa: reg = 2 – pior caso



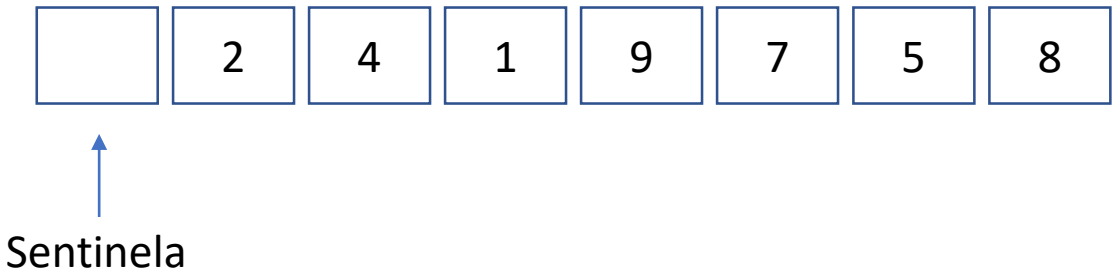
Pesquisa Sequencial

Chave pesquisa: reg = 2 – pior caso



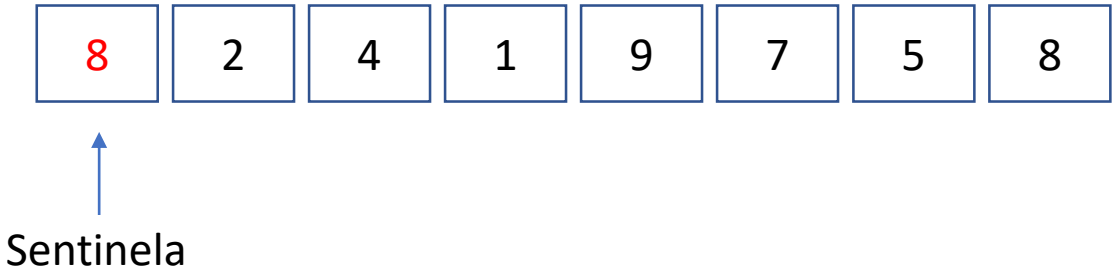
Pesquisa Sequencial

Chave pesquisa: reg = 8 – Melhor caso



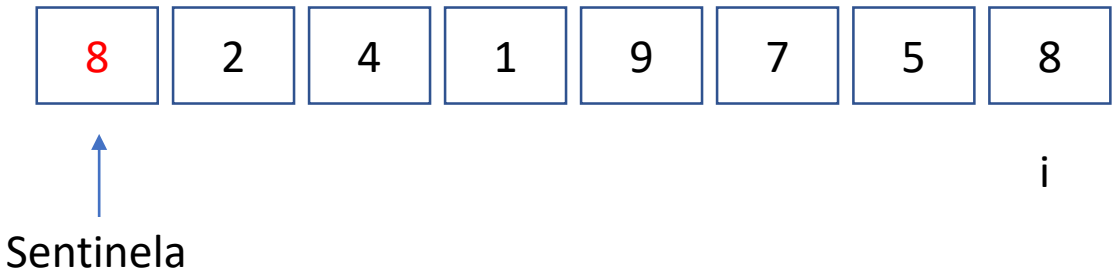
Pesquisa Sequencial

Chave pesquisa: reg = 8 – Melhor caso



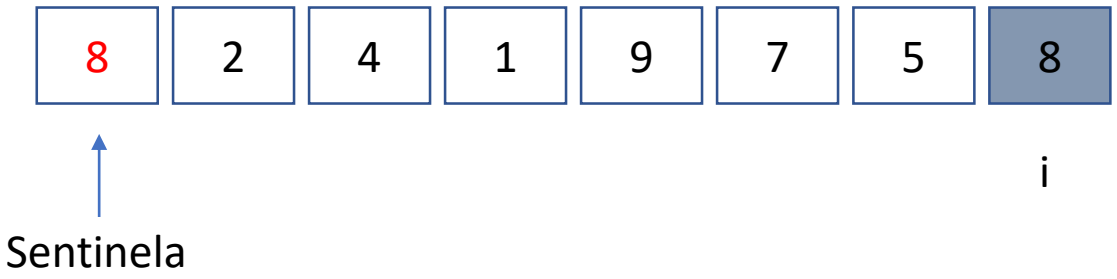
Pesquisa Sequencial

Chave pesquisa: reg = 8 – Melhor caso



Pesquisa Sequencial

Chave pesquisa: reg = 8 – Melhor caso



Pesquisa Sequencial

- Análise de custo:
  - Pesquisa com sucesso:
    - melhor caso: $C(n) = 1$
    - pior caso: $C(n) = n$
    - caso médio: $C(n) = (n + 1)/2$

  - Pesquisa sem sucesso:
    - $C(n) = n + 1$

Pesquisa Binária

- Pesquisa em tabela pode ser mais eficiente
  - Desde que registros sejam mantidos em ordem.
- Para saber se uma chave está presente na tabela:
  - Compare a chave com o registro que está na posição do meio da tabela.
  - **Se** a chave é menor **então** o registro procurado está na primeira metade da tabela.
  - **Se** a chave é maior **então** o registro procurado está na segunda metade da tabela.

Pesquisa Binária

- Repita o processo até que a chave seja encontrada, ou fique apenas um registro cuja chave é diferente da procurada, significando uma pesquisa sem sucesso.

Pesquisa Binária

Chave pesquisa G

0	1	2	3	4	5	6	7	8
	A	B	C	D	E	F	G	H
	esq							dir

Pesquisa Binária

Chave pesquisa G

0	1	2	3	4	5	6	7	8
	A	B	C	D	E	F	G	H
	esq			i				dir

Pesquisa Binária

Chave pesquisa G

0	1	2	3	4	5	6	7	8
	A	B	C	D	E	F	G	H
	esq			i				dir

Pesquisa Binária

Chave pesquisa G

0	1	2	3	4	5	6	7	8
	A	B	C	D	E	F	G	H
				i	esq			dir

Pesquisa Binária

Chave pesquisa G

0	1	2	3	4	5	6	7	8
	A	B	C	D	E	F	G	H
				i	esq			dir

Pesquisa Binária

Chave pesquisa G

0	1	2	3	4	5	6	7	8
	A	B	C	D	E	F	G	H
					esq	i		dir

Pesquisa Binária

Chave pesquisa G

0	1	2	3	4	5	6	7	8
	A	B	C	D	E	F	G	H
					esq	i		dir

Pesquisa Binária

Chave pesquisa G

0	1	2	3	4	5	6	7	8
	A	B	C	D	E	F	G	H
						i	esq	dir

Pesquisa Binária

Chave pesquisa G

0	1	2	3	4	5	6	7	8
	A	B	C	D	E	F	G	H
						i	esq	dir

Pesquisa Binária

Chave pesquisa G

0	1	2	3	4	5	6	7	8
	A	B	C	D	E	F	G	H
							esq	dir
							i	

Pesquisa Binária

Chave pesquisa G

0	1	2	3	4	5	6	7	8
	A	B	C	D	E	F	G	H
							esq	dir
							i	

Pesquisa Binária

Chave pesquisa G

0	1	2	3	4	5	6	7	8
	A	B	C	D	E	F	G	H
						dir	esq	
							i	

Pesquisa Binária

Chave pesquisa G

0	1	2	3	4	5	6	7	8
	A	B	C	D	E	F	G	H
						dir	esq	
							i	

Pesquisa Binária

Chave pesquisa G

0	1	2	3	4	5	6	7	8
	A	B	C	D	E	F	G	H
						dir	esq	
							i	

Pesquisa Binária

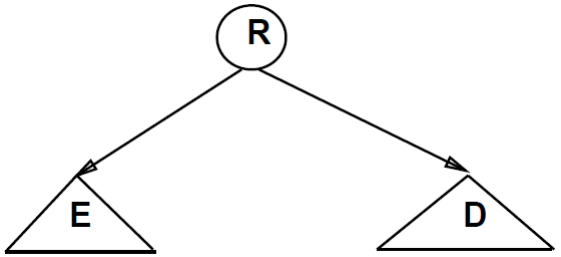
- Análise:
  - A cada iteração do algoritmo, o tamanho da tabela é dividido ao meio.
  - Número de vezes que o tamanho da tabela é dividido ao meio é cerca de *log n*.
  - Entretanto, o custo para manter a tabela ordenada é alto:
    - a cada inserção na posição p da tabela implica no deslocamento dos registros a partir da posição p para as posições seguintes.
  - Não deve ser usada para aplicações muito dinâmicas.

Árvores de Pesquisa

- Estrutura de dados muito eficiente para armazenar informação.
- Adequada quando existe necessidade de considerar todos ou alguma combinação de:
  - Acessos eficientes.
  - Facilidade de inserção e retirada de registros.
  - Boa taxa de utilização de memória.
  - Utilização de memória primária e secundária.

Árvores de Pesquisa

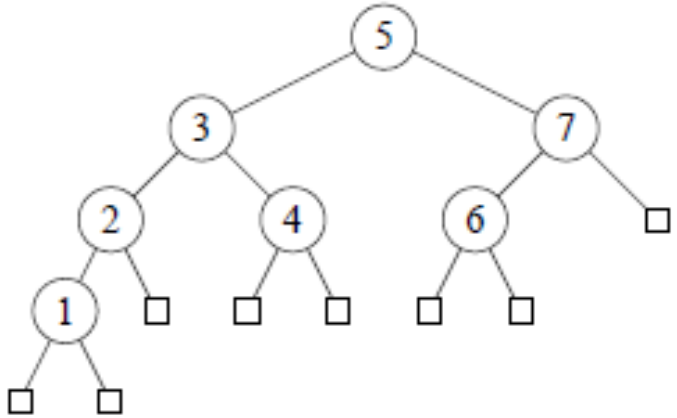
- Para qualquer nó que contenha um registro:



- Tem-se a relação invariante:

  - Todos os registros com chaves **menores** estão na sub árvore à **esquerda**.
  - Todos os registros com chaves **maiores** estão na sub árvore à **direita**.

Árvores de Pesquisa sem balanceamento



- O **nível** do nó raiz é 0.
- Se um nó está no **nível** i então a raiz de suas sub árvores estão no **nível** $i + 1$.
- A **altura** de um nó é o comprimento do caminho mais longo deste nó até um nó folha.
- A **altura** de uma árvore é a altura do nó raiz.

Árvores de Pesquisa – alguns conceitos

- Raiz da árvore: nó que não tem antecedente
- Folha: nós que não tem descendentes (os outros são os nós internos)
- A altura de um nó na árvore é o caminho de maior comprimento que se pode fazer deste nó a uma folha.
- A altura da árvore é a altura de sua raiz.
- A profundidade de um nó é o comprimento da raiz até o nó (só existe um caminho)

Implementação do TAD Dicionário usando Árvore Binária de Pesquisa

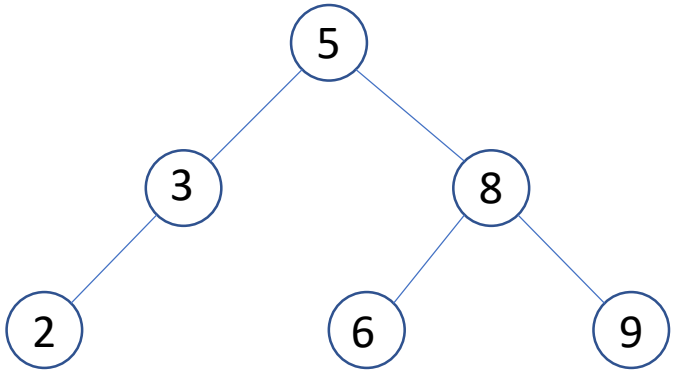
- Estrutura de dados:
  - Contém as operações
    - Inicializa
    - Pesquisa
    - Insere
    - retira.

Método para Pesquisar na Árvore

- Para encontrar um registro com uma chave *reg*:
  - Compare-a com a chave que está na raiz .
  - Se é menor, vá para a sub árvore esquerda.
  - Se é maior, vá para a sub árvore direita.
  - Repita o processo recursivamente, até que a chave procurada seja encontrada ou um nó folha é atingido.
  - Se a pesquisa tiver sucesso então o registro contendo a chave passada em *reg* é retornado.

Método para Pesquisar na Árvore

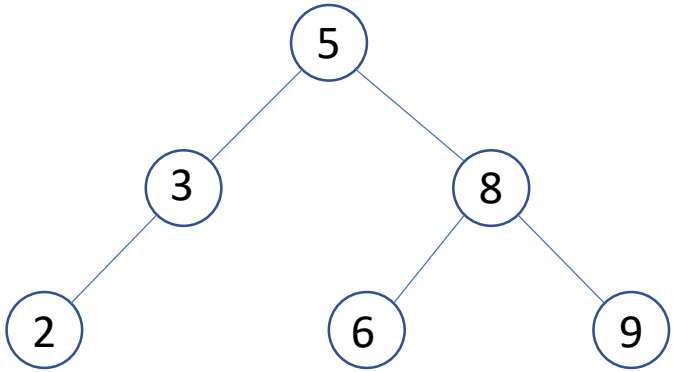
reg = 6




```
void Pesquisa(TipoRegistro *x, TipoApontador *p) {
    if (*p == NULL) {
        printf("Erro: Registro nao esta presente na arvore\n");
        return;
    }
    if (x->Chave < (*p)->Reg.Chave) {
        Pesquisa(x, &(*p)->Esq); return;
    }
    if (x->Chave > (*p)->Reg.Chave)
        Pesquisa(x, &(*p)->Dir);
    else *x = (*p)->Reg;
}
```

Método para Pesquisar na Árvore

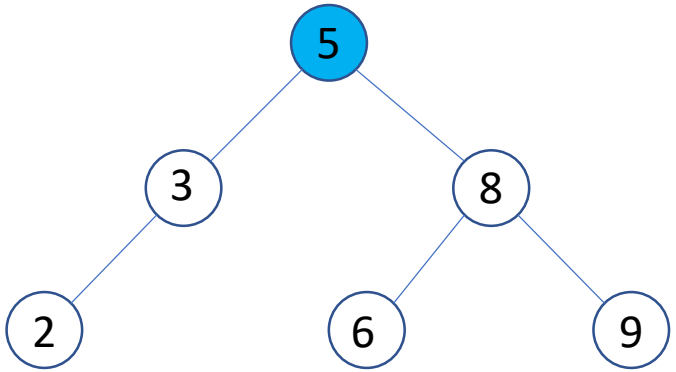
reg = 6




```
void Pesquisa(TipoRegistro *x, TipoApontador *p) {
    if (*p == NULL) {
        printf("Erro: Registro nao esta presente na arvore\n");
        return;
    }
    if (x->Chave < (*p)->Reg.Chave) {
        Pesquisa(x, &(*p)->Esq); return;
    }
    if (x->Chave > (*p)->Reg.Chave)
        Pesquisa(x, &(*p)->Dir);
    else *x = (*p)->Reg;
}
```

Método para Pesquisar na Árvore

reg = 6

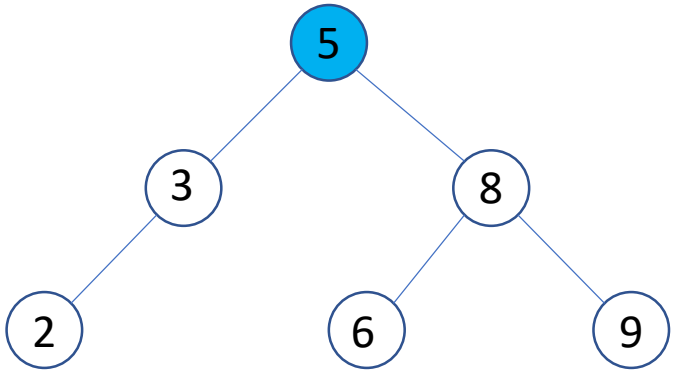



```
void Pesquisa(TipoRegistro *x, TipoApontador *p) {
    if (*p == NULL) {
        printf("Erro: Registro nao esta presente na arvore\n");
        return;
    }
    if (x->Chave < (*p)->Reg.Chave) {
        Pesquisa(x, &(*p)->Esq); return;
    }
    if (x->Chave > (*p)->Reg.Chave)
        Pesquisa(x, &(*p)->Dir);
    else *x = (*p)->Reg;
}
```

Compare reg com a chave que está na raiz. Se for menor, vá para a sub árvore esquerda.

Método para Pesquisar na Árvore

reg = 6

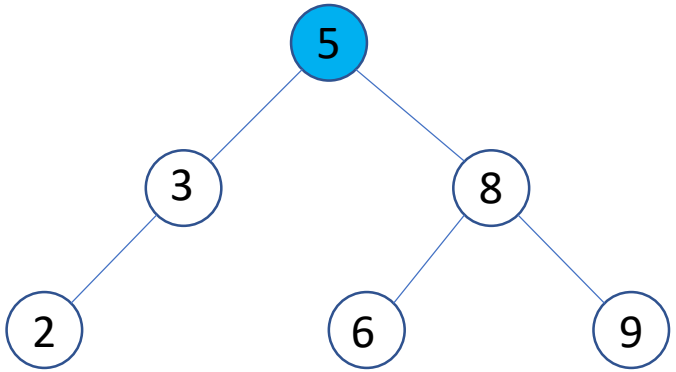



```
void Pesquisa(TipoRegistro *x, TipoApontador *p) {
    if (*p == NULL) {
        printf("Erro: Registro nao esta presente na arvore\n");
        return;
    }
    if (x->Chave < (*p)->Reg.Chave) {
        Pesquisa(x, &(*p)->Esq); return;
    }
    if (x->Chave > (*p)->Reg.Chave)
        Pesquisa(x, &(*p)->Dir);
    else *x = (*p)->Reg;
}
```

Se for maior, vá para a sub árvore direita.

Método para Pesquisar na Árvore

reg = 6

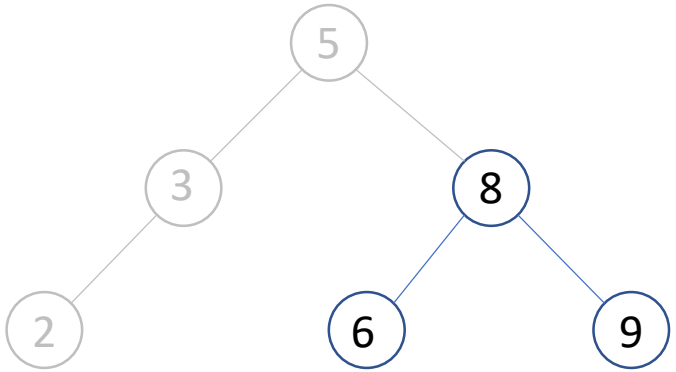



```
void Pesquisa(TipoRegistro *x, TipoApontador *p) {
    if (*p == NULL) {
        printf("Erro: Registro nao esta presente na arvore\n");
        return;
    }
    if (x->Chave < (*p)->Reg.Chave) {
        Pesquisa(x, &(*p)->Esq); return;
    }
    if (x->Chave > (*p)->Reg.Chave)
        Pesquisa(x, &(*p)->Dir);
    else *x = (*p)->Reg;
}
```

Repita o processo recursivamente, passando para a função apenas a sub árvore direita

Método para Pesquisar na Árvore

reg = 6

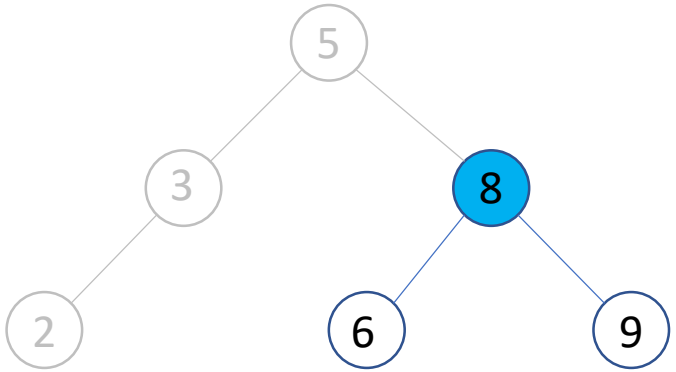



```
void Pesquisa(TipoRegistro *x, TipoApontador *p) {
    if (*p == NULL) {
        printf("Erro: Registro nao esta presente na arvore\n");
        return;
    }
    if (x->Chave < (*p)->Reg.Chave) {
        Pesquisa(x, &(*p)->Esq); return;
    }
    if (x->Chave > (*p)->Reg.Chave)
        Pesquisa(x, &(*p)->Dir);
    else *x = (*p)->Reg;
}
```

O No p é a sub árvore direita.

Método para Pesquisar na Árvore

reg = 6

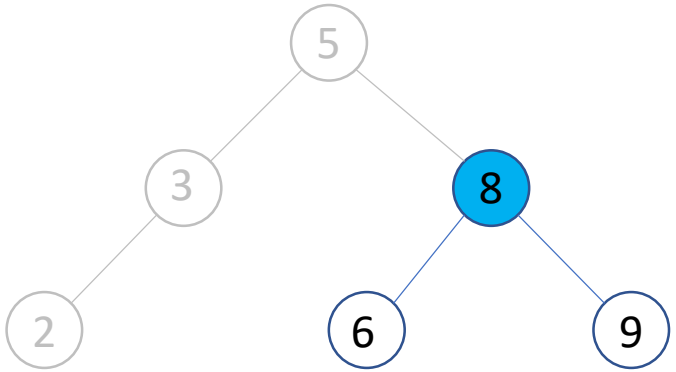



```
void Pesquisa(TipoRegistro *x, TipoApontador *p) {
    if (*p == NULL) {
        printf("Erro: Registro nao esta presente na arvore\n");
        return;
    }
    if (x->Chave < (*p)->Reg.Chave) {
        Pesquisa(x, &(*p)->Esq); return;
    }
    if (x->Chave > (*p)->Reg.Chave)
        Pesquisa(x, &(*p)->Dir);
    else *x = (*p)->Reg;
}
```

Compare reg com a chave que está na raiz. Se for menor, vá para a sub árvore esquerda.

Método para Pesquisar na Árvore

reg = 6

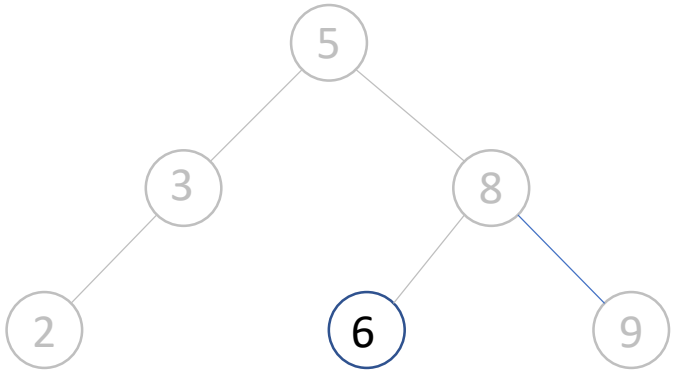



```
void Pesquisa(TipoRegistro *x, TipoApontador *p) {
    if (*p == NULL) {
        printf("Erro: Registro nao esta presente na arvore\n");
        return;
    }
    if (x->Chave < (*p)->Reg.Chave) {
        Pesquisa(x, &(*p)->Esq); return;
    }
    if (x->Chave > (*p)->Reg.Chave)
        Pesquisa(x, &(*p)->Dir);
    else *x = (*p)->Reg;
}
```

Repita o processo recursivamente, passando para a função apenas a sub árvore esquerda

Método para Pesquisar na Árvore

reg = 6

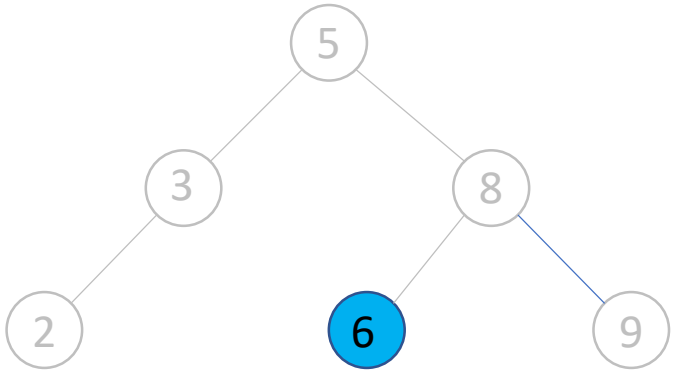



```
void Pesquisa(TipoRegistro *x, TipoApontador *p) {
    if (*p == NULL) {
        printf("Erro: Registro nao esta presente na arvore\n");
        return;
    }
    if (x->Chave < (*p)->Reg.Chave) {
        Pesquisa(x, &(*p)->Esq); return;
    }
    if (x->Chave > (*p)->Reg.Chave)
        Pesquisa(x, &(*p)->Dir);
    else *x = (*p)->Reg;
}
```

O No p é a sub árvore esquerda.

Método para Pesquisar na Árvore

reg = 6

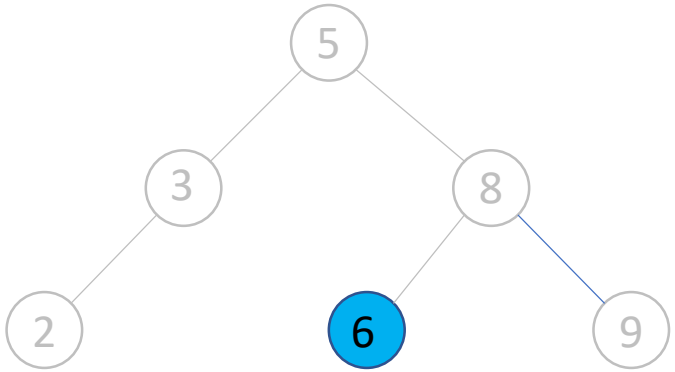



```
void Pesquisa(TipoRegistro *x, TipoApontador *p) {
    if (*p == NULL) {
        printf("Erro: Registro nao esta presente na arvore\n");
        return;
    }
    if (x->Chave < (*p)->Reg.Chave) {
        Pesquisa(x, &(*p)->Esq); return;
    }
    if (x->Chave > (*p)->Reg.Chave)
        Pesquisa(x, &(*p)->Dir);
    else *x = (*p)->Reg;
}
```

Compare reg com a chave que está na raiz. Se for menor, vá para a sub árvore esquerda.

Método para Pesquisar na Árvore

reg = 6

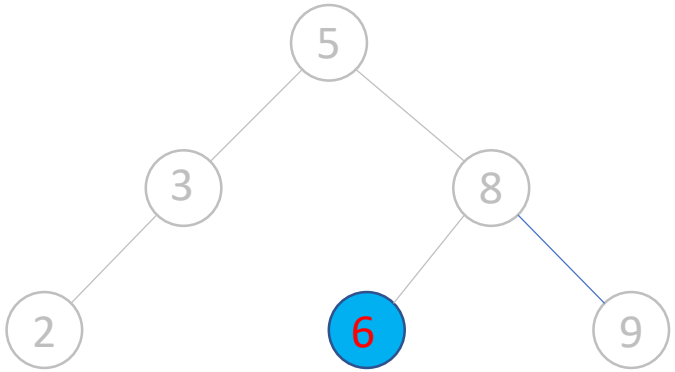



```
void Pesquisa(TipoRegistro *x, TipoApontador *p) {
    if (*p == NULL) {
        printf("Erro: Registro nao esta presente na arvore\n");
        return;
    }
    if (x->Chave < (*p)->Reg.Chave) {
        Pesquisa(x, &(*p)->Esq); return;
    }
    if (x->Chave > (*p)->Reg.Chave)
        Pesquisa(x, &(*p)->Dir);
    else *x = (*p)->Reg;
}
```

Compare reg com a chave que está na raiz. Se for maior, vá para a sub árvore direita.

Método para Pesquisar na Árvore

reg = 6




```
void Pesquisa(TipoRegistro *x, TipoApontador *p) {
    if (*p == NULL) {
        printf("Erro: Registro nao esta presente na arvore\n");
        return;
    }
    if (x->Chave < (*p)->Reg.Chave) {
        Pesquisa(x, &(*p)->Esq); return;
    }
    if (x->Chave > (*p)->Reg.Chave)
        Pesquisa(x, &(*p)->Dir);
    else *x = (*p)->Reg;
}
```

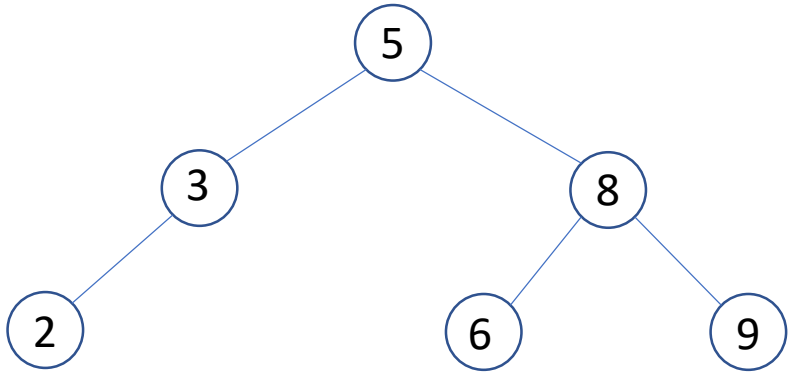
Pesquisa obteve sucesso: retorne o registro contendo a chave reg.

Procedimento para Inserir na Árvore

- Ao pesquisar por um registro, atingir uma referência **null** em um processo de pesquisa significa uma pesquisa sem sucesso.
- Caso queira inseri-lo na árvore, a referência **null** atingida é justamente o ponto de inserção.

Procedimento para Inserir na Árvore

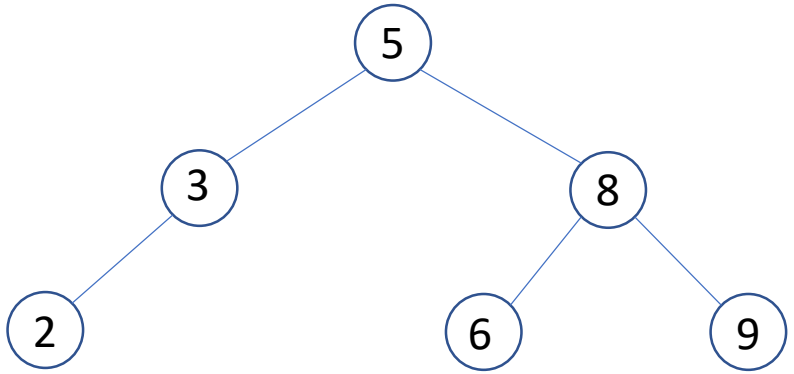
reg = 4




```
void Insere(TipoRegistro x, TipoApontador *p) {
    if (*p == NULL) {
        *p = (TipoApontador)malloc(sizeof(TipoNo));
        (*p)->Reg = x;
        (*p)->Esq = NULL;
        (*p)->Dir = NULL;
        return;
    }
    if (x.Chave < (*p)->Reg.Chave) {
        Insere(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave)
        Insere(x, &(*p)->Dir);
    else
        printf("Erro : Registro ja existe na arvore\n");
}
```

Procedimento para Inserir na Árvore

reg = 4

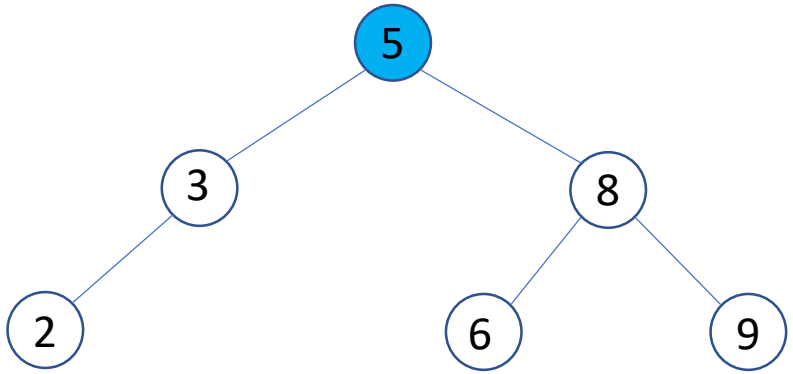



```
void Insere(TipoRegistro x, TipoApontador *p) {
    if (*p == NULL) {
        *p = (TipoApontador)malloc(sizeof(TipoNo));
        (*p)->Reg = x;
        (*p)->Esq = NULL;
        (*p)->Dir = NULL;
        return;
    }
    if (x.Chave < (*p)->Reg.Chave) {
        Insere(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave)
        Insere(x, &(*p)->Dir);
    else
        printf("Erro : Registro ja existe na arvore\n");
}
```

Se p = null, árvore está vazia, então cria-se um novo No p para a árvore, que conterá apenas um elemento.

Procedimento para Inserir na Árvore

reg = 4

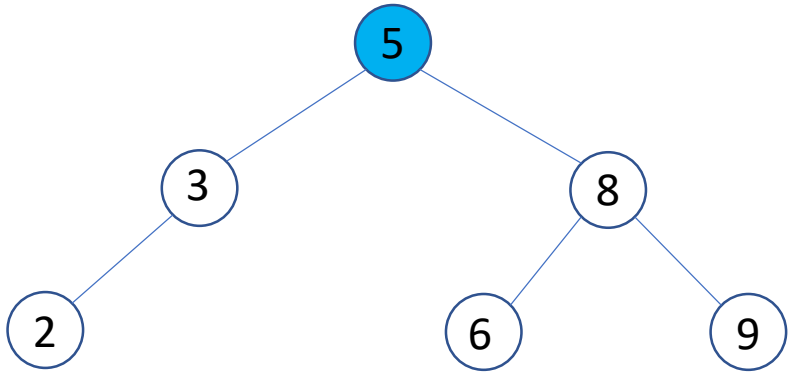



```
void Insere(TipoRegistro x, TipoApontador *p) {
    if (*p == NULL) {
        *p = (TipoApontador)malloc(sizeof(TipoNo));
        (*p)->Reg = x;
        (*p)->Esq = NULL;
        (*p)->Dir = NULL;
        return;
    }
    if (x.Chave < (*p)->Reg.Chave) {
        Insere(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave)
        Insere(x, &(*p)->Dir);
    else
        printf("Erro : Registro ja existe na arvore\n");
}
```

Compare reg com a chave que está na raiz. Se for menor, vá para a sub árvore esquerda.

Procedimento para Inserir na Árvore

reg = 4

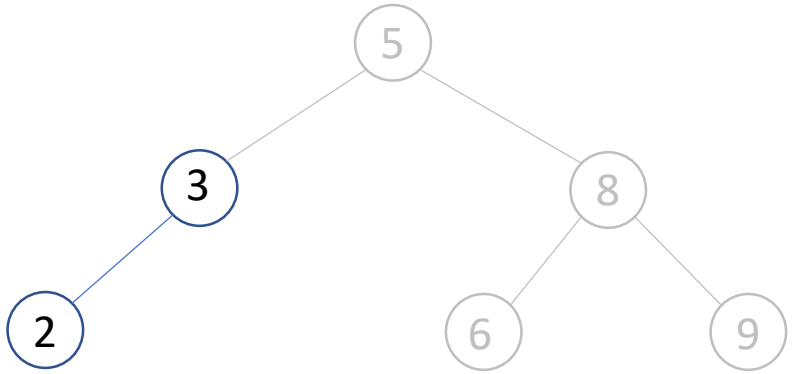



```
void Insere(TipoRegistro x, TipoApontador *p) {
    if (*p == NULL) {
        *p = (TipoApontador)malloc(sizeof(TipoNo));
        (*p)->Reg = x;
        (*p)->Esq = NULL;
        (*p)->Dir = NULL;
        return;
    }
    if (x.Chave < (*p)->Reg.Chave) {
        Insere(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave)
        Insere(x, &(*p)->Dir);
    else
        printf("Erro : Registro ja existe na arvore\n");
}
```

Repita o processo recursivamente, passando para a função apenas a sub árvore esquerda

Procedimento para Inserir na Árvore

reg = 4

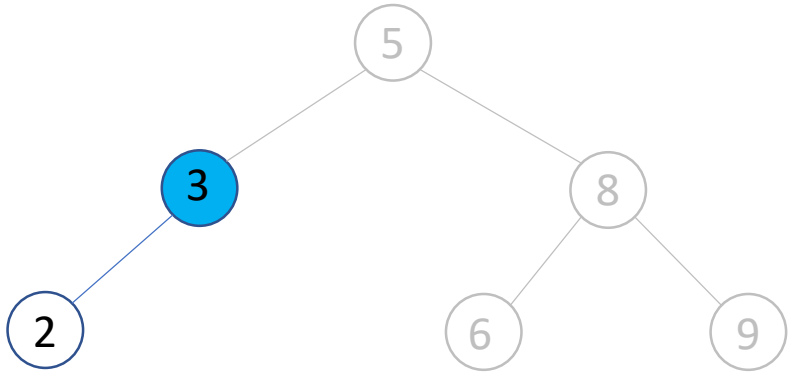



```
void Insere(TipoRegistro x, TipoApontador *p) {
    if (*p == NULL) {
        *p = (TipoApontador)malloc(sizeof(TipoNo));
        (*p)->Reg = x;
        (*p)->Esq = NULL;
        (*p)->Dir = NULL;
        return;
    }
    if (x.Chave < (*p)->Reg.Chave) {
        Insere(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave)
        Insere(x, &(*p)->Dir);
    else
        printf("Erro : Registro ja existe na arvore\n");
}
```

Se p = null, árvore está vazia, então cria-se um novo No p para a árvore, que conterá apenas um elemento.

Procedimento para Inserir na Árvore

reg = 4

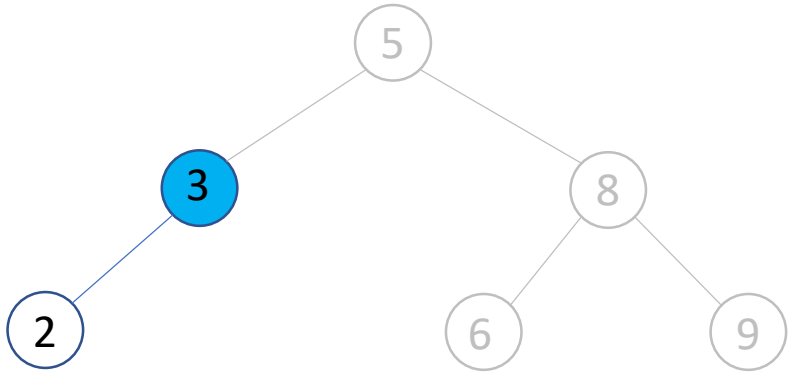



```
void Insere(TipoRegistro x, TipoApontador *p) {
    if (*p == NULL) {
        *p = (TipoApontador)malloc(sizeof(TipoNo));
        (*p)->Reg = x;
        (*p)->Esq = NULL;
        (*p)->Dir = NULL;
        return;
    }
    if (x.Chave < (*p)->Reg.Chave) {
        Insere(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave)
        Insere(x, &(*p)->Dir);
    else
        printf("Erro : Registro ja existe na arvore\n");
}
```

Compare reg com a chave que está na raiz. Se for menor, vá para a sub árvore esquerda.

Procedimento para Inserir na Árvore

reg = 4

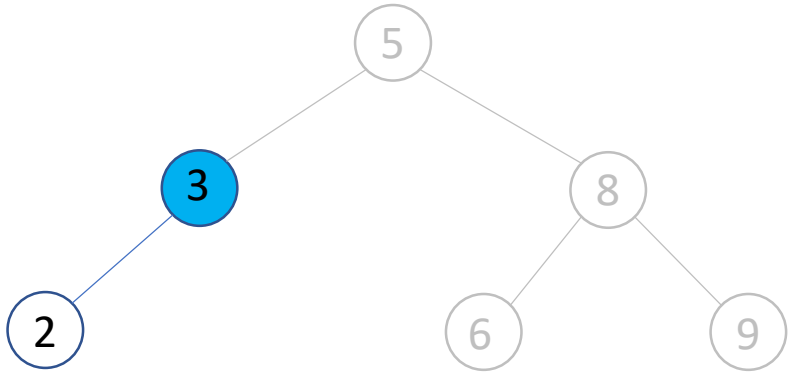



```
void Insere(TipoRegistro x, TipoApontador *p) {
    if (*p == NULL) {
        *p = (TipoApontador)malloc(sizeof(TipoNo));
        (*p)->Reg = x;
        (*p)->Esq = NULL;
        (*p)->Dir = NULL;
        return;
    }
    if (x.Chave < (*p)->Reg.Chave) {
        Insere(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave)
        Insere(x, &(*p)->Dir);
    else
        printf("Erro : Registro ja existe na arvore\n");
}
```

Compare reg com a chave que está na raiz. Se for maior, vá para a sub árvore direita.

Procedimento para Inserir na Árvore

reg = 4

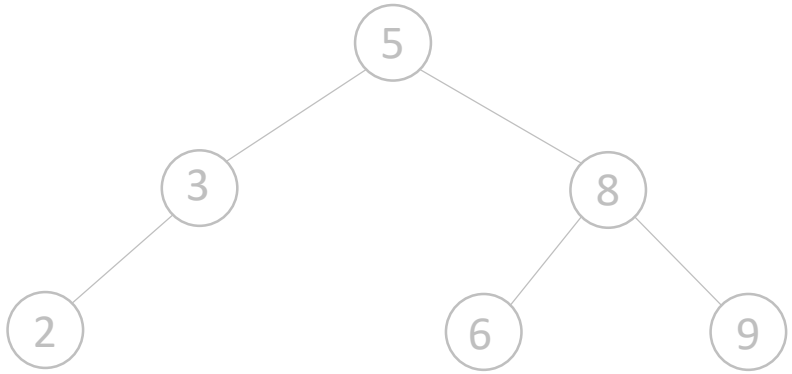



```
void Insere(TipoRegistro x, TipoApontador *p) {
    if (*p == NULL) {
        *p = (TipoApontador)malloc(sizeof(TipoNo));
        (*p)->Reg = x;
        (*p)->Esq = NULL;
        (*p)->Dir = NULL;
        return;
    }
    if (x.Chave < (*p)->Reg.Chave) {
        Insere(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave)
        Insere(x, &(*p)->Dir);
    else
        printf("Erro : Registro ja existe na arvore\n");
}
```

Repita o processo recursivamente, passando para a função apenas a sub árvore direita.
Obs.: Neste momento, p.dir = null

Procedimento para Inserir na Árvore

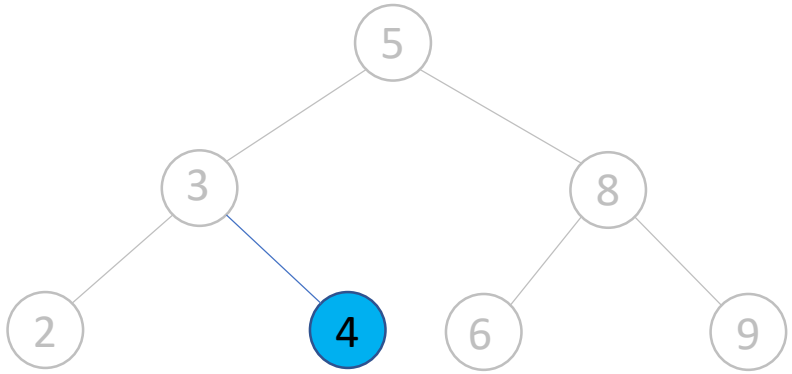
reg = 4




```
void Insere(TipoRegistro x, TipoApontador *p) {
    if (*p == NULL) {
        *p = (TipoApontador)malloc(sizeof(TipoNo));
        (*p)->Reg = x;
        (*p)->Esq = NULL;
        (*p)->Dir = NULL;
        return;
    }
    if (x.Chave < (*p)->Reg.Chave) {
        Insere(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave)
        Insere(x, &(*p)->Dir);
    else
        printf("Erro : Registro ja existe na arvore\n");
}
```

Procedimento para Inserir na Árvore

reg = 4

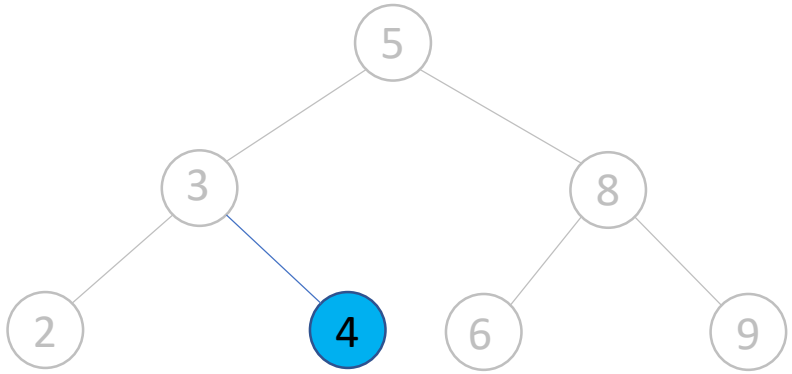



```
void Insere(TipoRegistro x, TipoApontador *p) {
    if (*p == NULL) {
        *p = (TipoApontador)malloc(sizeof(TipoNo));
        (*p)->Reg = x;
        (*p)->Esq = NULL;
        (*p)->Dir = NULL;
        return;
    }
    if (x.Chave < (*p)->Reg.Chave) {
        Insere(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave)
        Insere(x, &(*p)->Dir);
    else
        printf("Erro : Registro ja existe na arvore\n");
}
```

Cria-se um novo nó, e insere o registro nele

Procedimento para Inserir na Árvore

reg = 4

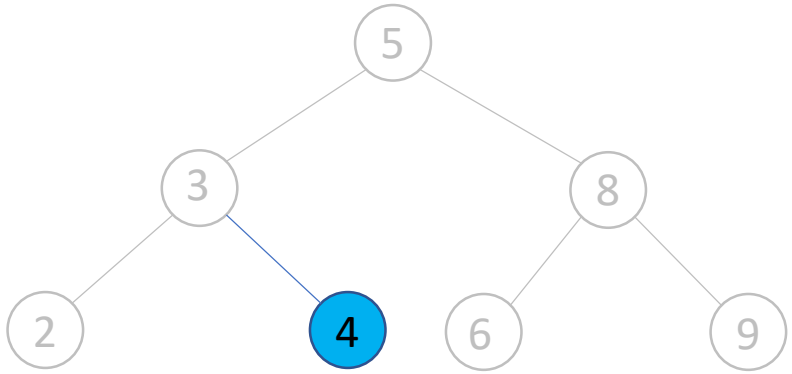



```
void Insere(TipoRegistro x, TipoApontador *p) {
    if (*p == NULL) {
        *p = (TipoApontador)malloc(sizeof(TipoNo));
        (*p)->Reg = x;
        (*p)->Esq = NULL;
        (*p)->Dir = NULL;
        return;
    }
    if (x.Chave < (*p)->Reg.Chave) {
        Insere(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave)
        Insere(x, &(*p)->Dir);
    else
        printf("Erro : Registro ja existe na arvore\n");
}
```

Como o novo nó inserido não tem nós filhos, as sub árvores esquerda e direita serão nulas

Procedimento para Inserir na Árvore

reg = 4

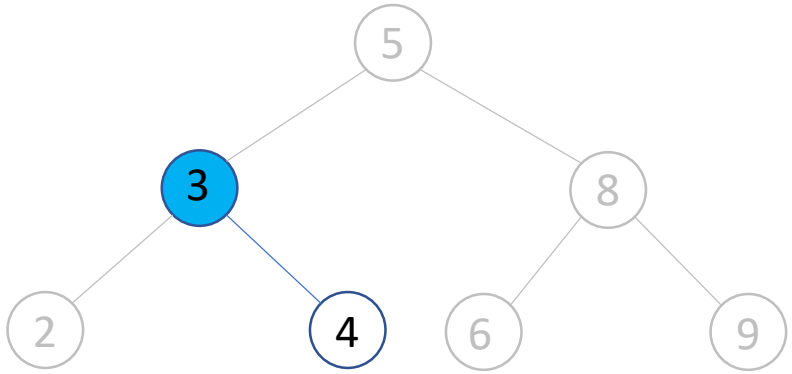



```
void Insere(TipoRegistro x, TipoApontador *p) {
    if (*p == NULL) {
        *p = (TipoApontador)malloc(sizeof(TipoNo));
        (*p)->Reg = x;
        (*p)->Esq = NULL;
        (*p)->Dir = NULL;
        return;
    }
    if (x.Chave < (*p)->Reg.Chave) {
        Insere(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave)
        Insere(x, &(*p)->Dir);
    else
        printf("Erro : Registro ja existe na arvore\n");
}
```

Retorno para a função.

Procedimento para Inserir na Árvore

reg = 4



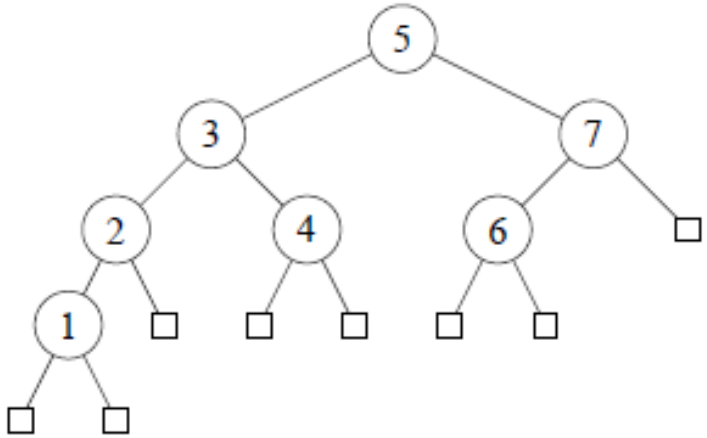

```
void Insere(TipoRegistro x, TipoApontador *p) {
    if (*p == NULL) {
        *p = (TipoApontador)malloc(sizeof(TipoNo));
        (*p)->Reg = x;
        (*p)->Esq = NULL;
        (*p)->Dir = NULL;
        return;
    }
    if (x.Chave < (*p)->Reg.Chave) {
        Insere(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave)
        Insere(x, &(*p)->Dir);
    else
        printf("Erro : Registro ja existe na arvore\n");
}
```

A sub árvore direita do nó (3) recebe o novo nó (4) como nó filho.

Procedimento para Retirar um elemento *x* da Árvore

- A retirada de um registro não é tão simples quanto a inserção.
- Se o nó que contém o registro a ser retirado possui no máximo um descendente: a operação é simples.
- No caso do nó conter dois descendentes o registro a ser retirado deve ser primeiro:
  - substituído pelo **registro mais à direita na sub árvore esquerda**;
  - ou pelo **registro mais à esquerda na sub árvore direita**.

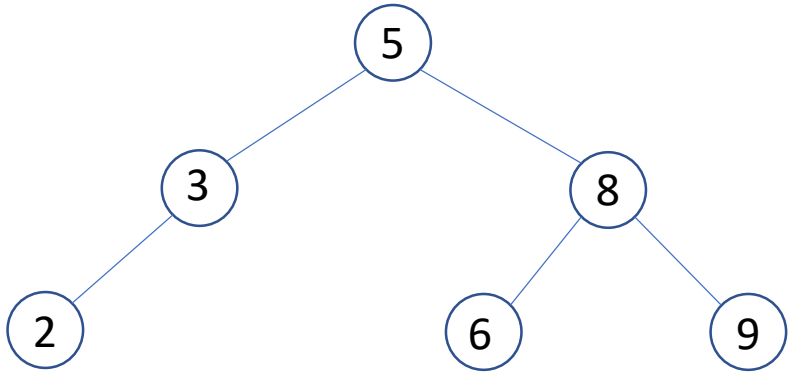
Procedimento para Retirar um elemento *x* da Árvore



- Retirar o registro com chave 5:
  - trocá-lo pelo registro com chave 4 (**registro mais à direita na sub árvore esquerda**) ou pelo registro com chave 6 (**registro mais à esquerda na sub árvore direita**), e então retirar o nó que recebeu o registro com chave 5

Procedimento para Retirar um elemento *x* da Árvore

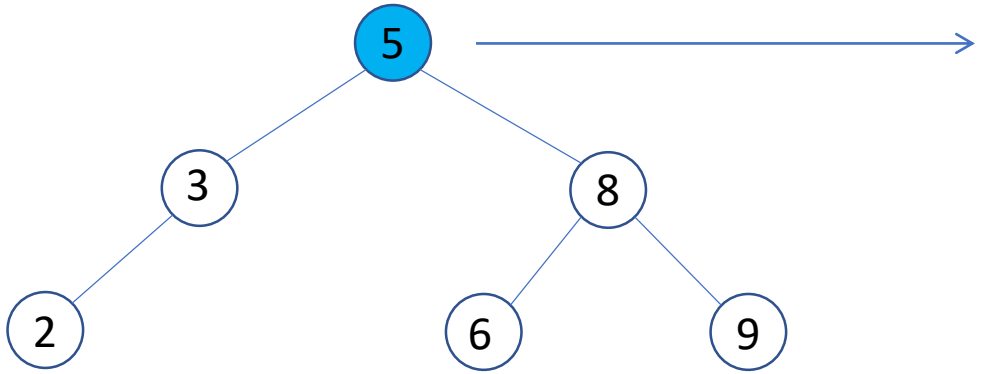
reg = 5




```
void Retira(TipoRegistro x, TipoApontador *p) {
    ...
    if (x.Chave < (*p)->Reg.Chave) {
        Retira(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave) {
        Retira(x, &(*p)->Dir); return;
    }
    if ((*p)->Dir == NULL) {
        Aux = *p; *p = (*p)->Esq; free(Aux); return;
    }
    if ((*p)->Esq != NULL) {
        Antecessor(*p, &(*p)->Esq); return;
    }
    Aux = *p; *p = (*p)->Dir; free(Aux);
}
```

Procedimento para Retirar um elemento *x* da Árvore

reg = 5

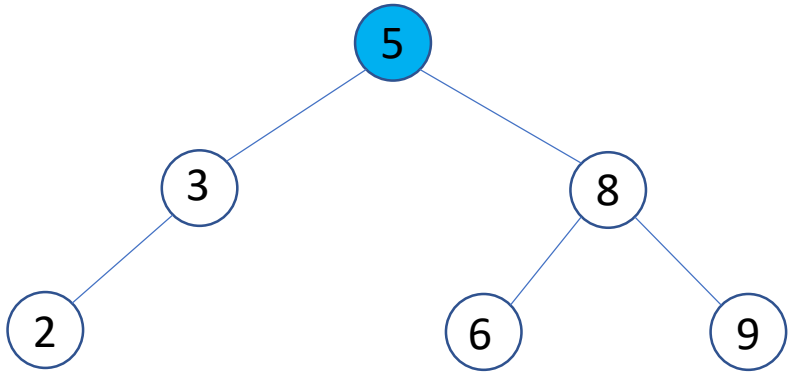


Pesquisa o elemento a ser removido na árvore

```
void Retira(TipoRegistro x, TipoApontador *p) {
    ...
    if (x.Chave < (*p)->Reg.Chave) {
        Retira(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave) {
        Retira(x, &(*p)->Dir); return;
    }
    if ((*p)->Dir == NULL) {
        Aux = *p; *p = (*p)->Esq; free(Aux); return;
    }
    if ((*p)->Esq != NULL) {
        Antecessor(*p, &(*p)->Esq); return;
    }
    Aux = *p; *p = (*p)->Dir; free(Aux);
}
```

Procedimento para Retirar um elemento *x* da Árvore

reg = 5

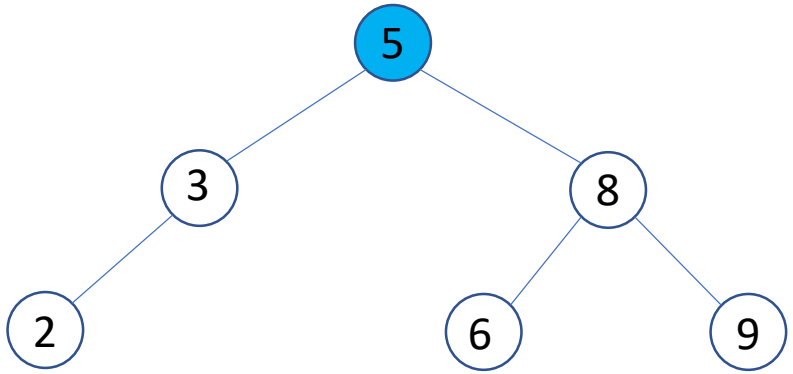



```
void Retira(TipoRegistro x, TipoApontador *p) {
    ...
    if (x.Chave < (*p)->Reg.Chave) {
        Retira(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave) {
        Retira(x, &(*p)->Dir); return;
    }
    if ((*p)->Dir == NULL) {
        Aux = *p; *p = (*p)->Esq; free(Aux); return;
    }
    if ((*p)->Esq != NULL) {
        Antecessor(*p, &(*p)->Esq); return;
    }
    Aux = *p; *p = (*p)->Dir; free(Aux);
}
```

Se não existir a sub árvore dir, a raiz da sub árvore esq passa a ser a raiz da árvore.

Procedimento para Retirar um elemento *x* da Árvore

reg = 5

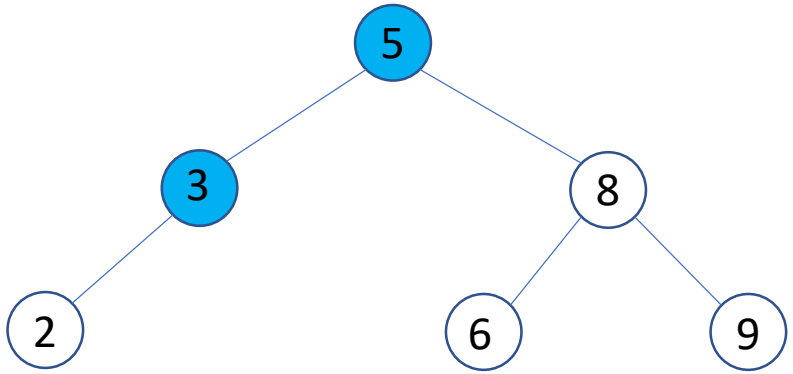



```
void Retira(TipoRegistro x, TipoApontador *p) {
    ...
    if (x.Chave < (*p)->Reg.Chave) {
        Retira(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave) {
        Retira(x, &(*p)->Dir); return;
    }
    if ((*p)->Dir == NULL) {
        Aux = *p; *p = (*p)->Esq; free(Aux); return;
    }
    if ((*p)->Esq != NULL) {
        Antecessor(*p, &(*p)->Esq); return;
    }
    Aux = *p; *p = (*p)->Dir; free(Aux);
}
```

Se não existir a sub árvore esq, a raiz da sub árvore dir passa a ser a raiz da árvore.

Procedimento para Retirar um elemento *x* da Árvore

reg = 5

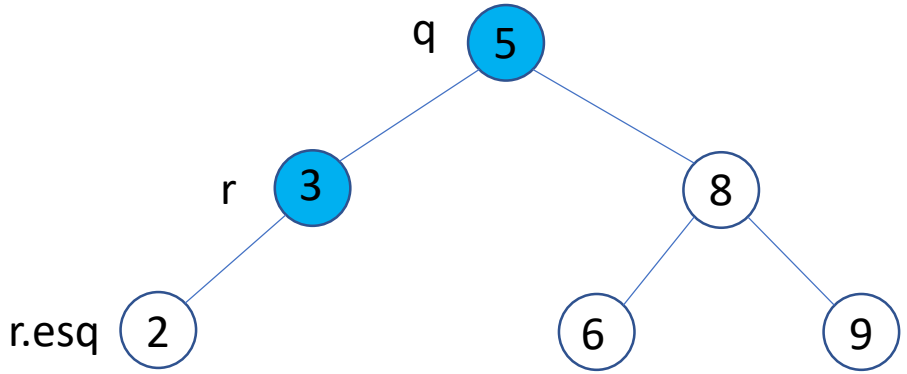



```
void Retira(TipoRegistro x, TipoApontador *p) {
    ...
    if (x.Chave < (*p)->Reg.Chave) {
        Retira(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave) {
        Retira(x, &(*p)->Dir); return;
    }
    if ((*p)->Dir == NULL) {
        Aux = *p; *p = (*p)->Esq; free(Aux); return;
    }
    if ((*p)->Esq != NULL) {
        Antecessor(*p, &(*p)->Esq); return;
    }
    Aux = *p; *p = (*p)->Dir; free(Aux);
}
```

Caso exista sub árvore esq e dir, calcular o elemento mais a direta da sub árvore esquerda de p

Procedimento para Retirar um elemento *x* da Árvore

reg = 5

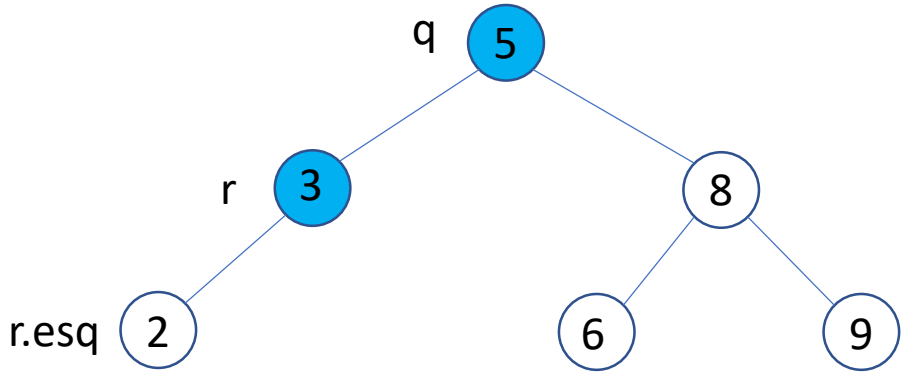


Se existe uma sub árvore direita, continuar pesquisa até encontrar o elemento mais a direita

```
void Antecessor(TipoApontador q, TipoApontador *r) {
    if ((*r)->Dir != NULL) {
        Antecessor(q, &(*r)->Dir); return;
    }
    q->Reg = (*r)->Reg;
    q = *r;
    *r = (*r)->Esq;
    free(q);
}
```

Procedimento para Retirar um elemento *x* da Árvore

reg = 5

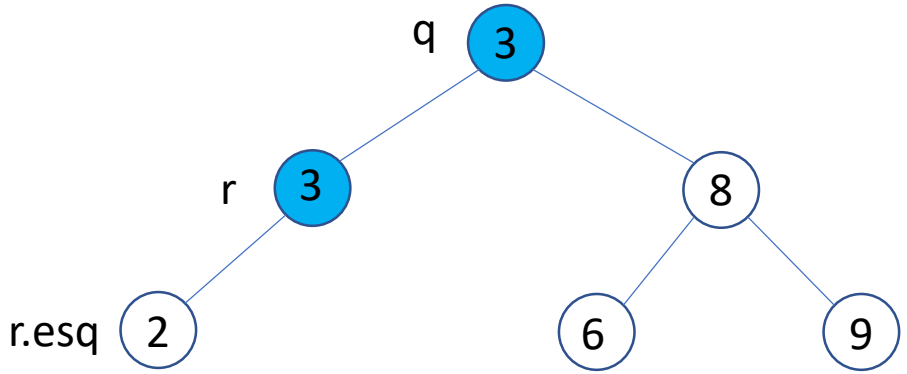


Se existe uma sub árvore direita, continuar pesquisa até encontrar o elemento mais a direita

```
void Antecessor(TipoApontador q, TipoApontador *r) {
    if ((*r)->Dir != NULL) {
        Antecessor(q, &(*r)->Dir); return;
    }
    q->Reg = (*r)->Reg;
    q = *r;
    *r = (*r)->Esq;
    free(q);
}
```

Procedimento para Retirar um elemento *x* da Árvore

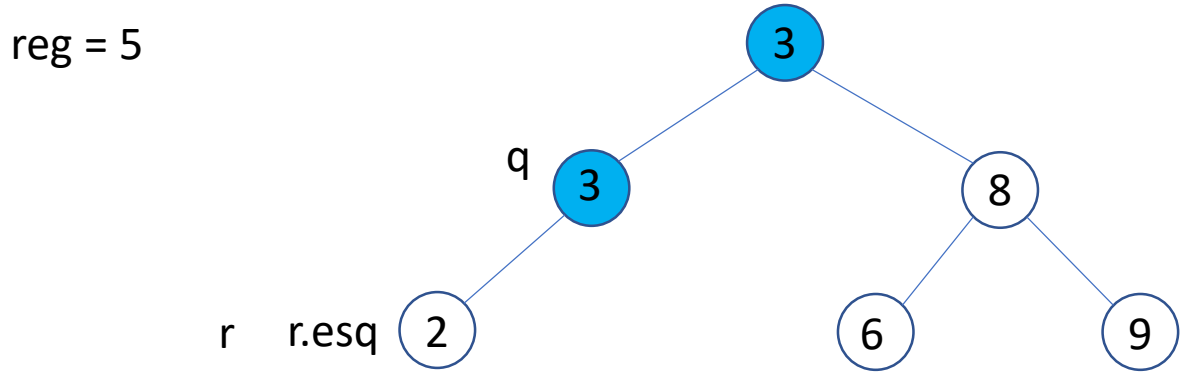
reg = 5



Se existe uma sub árvore direita, continuar pesquisa até encontrar o elemento mais a direita

```
void Antecessor(TipoApontador q, TipoApontador *r) {
    if ((*r)->Dir != NULL) {
        Antecessor(q, &(*r)->Dir); return;
    }
    q->Reg = (*r)->Reg;
    q = *r;
    *r = (*r)->Esq;
    free(q);
}
```

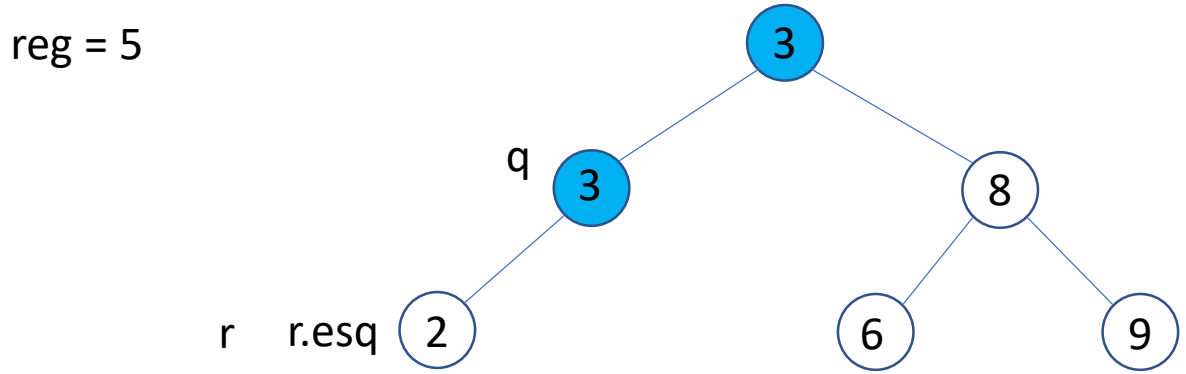
Procedimento para Retirar um elemento *x* da Árvore



Ao encontrar o elemento mais a direita, q recebe r (o elemento mais a direita), e r recebe sua sub árvore a esquerda

```
void Antecessor(TipoApontador q, TipoApontador *r) {
    if ((*r)->Dir != NULL) {
        Antecessor(q, &(*r)->Dir); return;
    }
    q->Reg = (*r)->Reg;
    q = *r;
    *r = (*r)->Esq;
    free(q);
}
```

Procedimento para Retirar um elemento *x* da Árvore

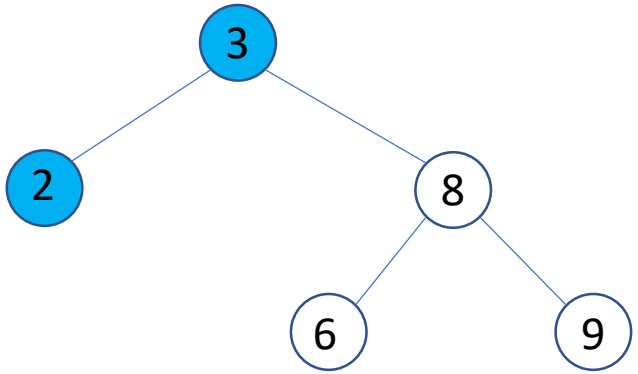



```
void Antecessor(TipoApontador q, TipoApontador *r) {
    if ((*r)->Dir != NULL) {
        Antecessor(q, &(*r)->Dir); return;
    }
    q->Reg = (*r)->Reg;
    q = *r;
    *r = (*r)->Esq;
    free(q);
}
```

Retorna a sub árvore a esquerda

Procedimento para Retirar um elemento *x* da Árvore

reg = 5

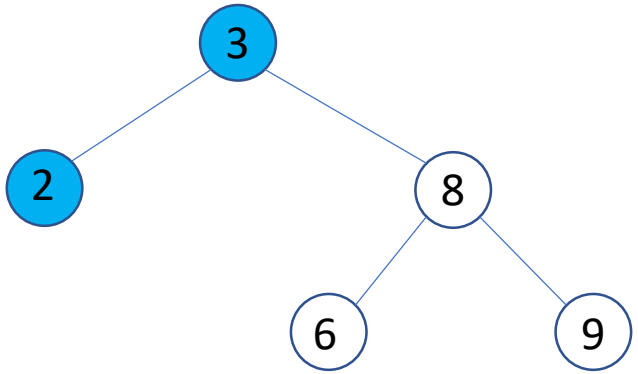



```
void Retira(TipoRegistro x, TipoApontador *p) {
    ...
    if (x.Chave < (*p)->Reg.Chave) {
        Retira(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave) {
        Retira(x, &(*p)->Dir); return;
    }
    if ((*p)->Dir == NULL) {
        Aux = *p; *p = (*p)->Esq; free(Aux); return;
    }
    if ((*p)->Esq != NULL) {
        Antecessor(*p, &(*p)->Esq); return;
    }
    Aux = *p; *p = (*p)->Dir; free(Aux);
}
```

```
void Antecessor(TipoApontador q, TipoApontador *r) {
    if ((*r)->Dir != NULL) {
        Antecessor(q, &(*r)->Dir); return;
    }
    q->Reg = (*r)->Reg;
    q = *r;
    *r = (*r)->Esq;
    free(q);
}
```

Procedimento para Retirar um elemento *x* da Árvore

reg = 5




```
void Retira(TipoRegistro x, TipoApontador *p) {
    ...
    if (x.Chave < (*p)->Reg.Chave) {
        Retira(x, &(*p)->Esq); return;
    }
    if (x.Chave > (*p)->Reg.Chave) {
        Retira(x, &(*p)->Dir); return;
    }
    if ((*p)->Dir == NULL) {
        Aux = *p; *p = (*p)->Esq; free(Aux); return;
    }
    if ((*p)->Esq != NULL) {
        Antecessor(*p, &(*p)->Esq); return;
    }
    Aux = *p; *p = (*p)->Dir; free(Aux);
}
```

Retorna a nova raiz da árvore

```
void Antecessor(TipoApontador q, TipoApontador *r) {
    if ((*r)->Dir != NULL) {
        Antecessor(q, &(*r)->Dir); return;
    }
    q->Reg = (*r)->Reg;
    q = *r;
    *r = (*r)->Esq;
    free(q);
}
```

Caminhamento Central (ou Em-ordem)

- Após construída a árvore, pode ser necessário percorrer todos os registros que compõem a tabela ou arquivo.
- Existe mais de uma ordem de **caminhamento** em árvores, mas a mais útil é a chamada ordem de **caminhamento central**.
- O caminhamento central é melhor expresso em termos recursivos.

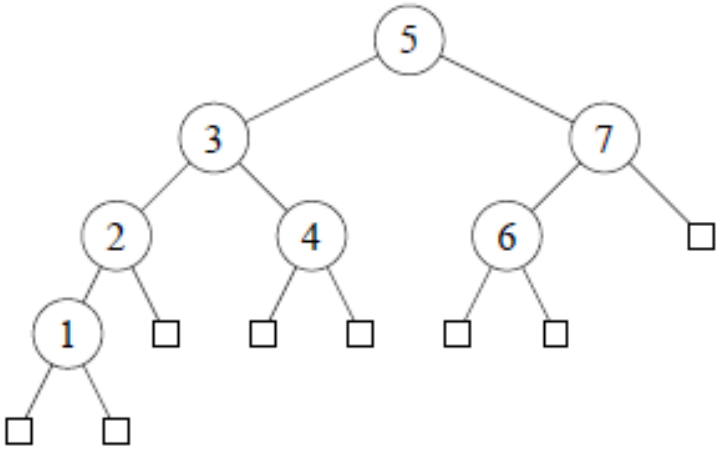
Caminhamento Central

- O caminhamento central lista o **nó filho da subárvore mais à esquerda**, lista o **nó raiz** desta subárvore e então o **nó filho à direita**. (em cada subárvore lista os nós sempre da esquerda para a direita). O procedimento é aplicado recursivamente a toda árvore.
  - Em resumo: os nós de cada sub árvore são listados nesta ordem: **nós filhos à esquerda**, **raiz** e **nós filhos à direita**.

- Uma característica importante do caminhamento central é que os nós são visitados de **forma ordenada**.

Caminhamento Central

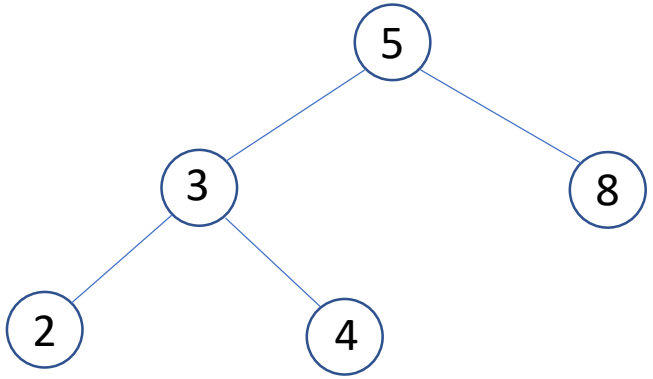
- Percorrer a árvore:



- Usando caminhamento central recupera as chaves na ordem 1, 2, 3, 4, 5, 6 e 7.

Caminhamento Central

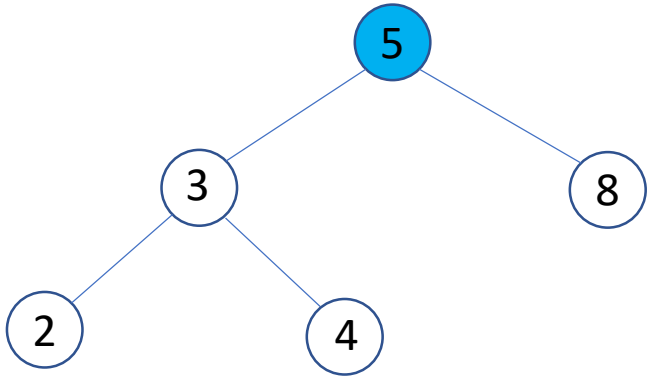
- Caminhamento central e impressão da árvore:




```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

Caminhamento Central

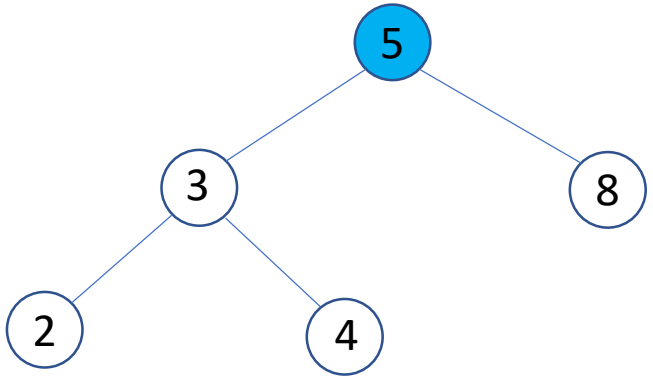
- Caminhamento central e impressão da árvore:




```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

Caminhamento Central

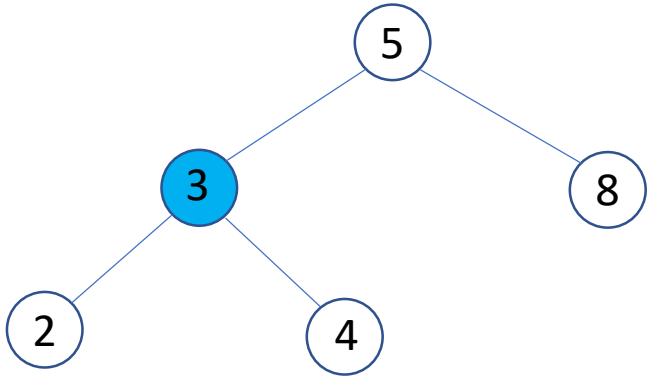
- Caminhamento central e impressão da árvore:




```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

Caminhamento Central

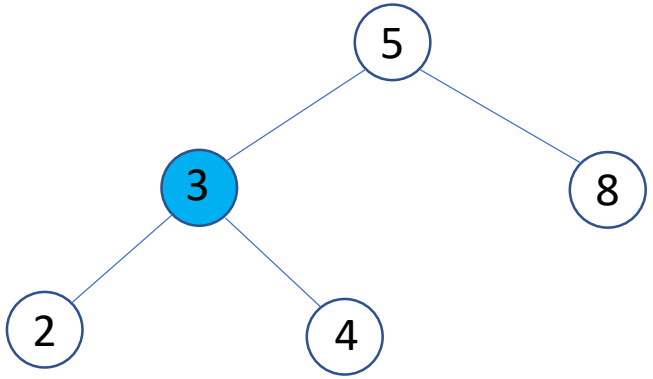
- Caminhamento central e impressão da árvore:




```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

Caminhamento Central

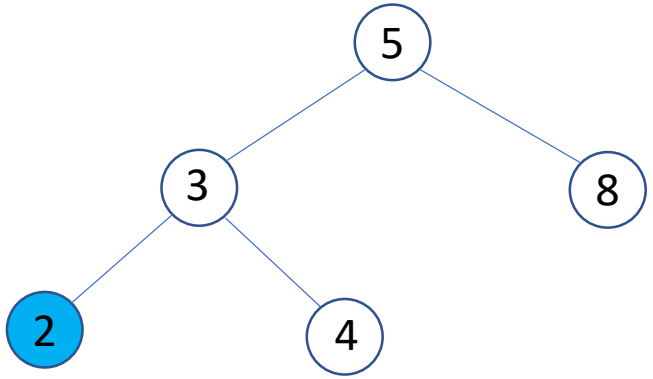
- Caminhamento central e impressão da árvore:




```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

Caminhamento Central

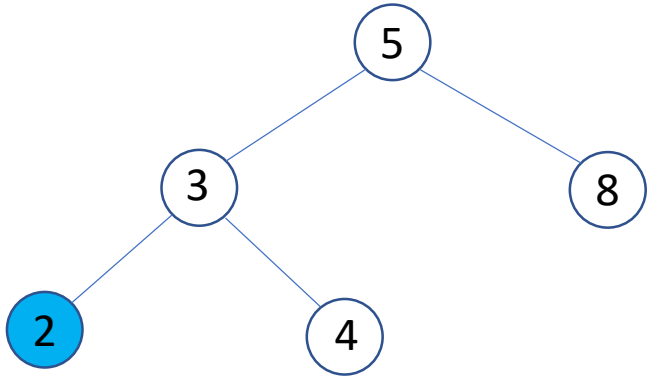
- Caminhamento central e impressão da árvore:




```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

Caminhamento Central

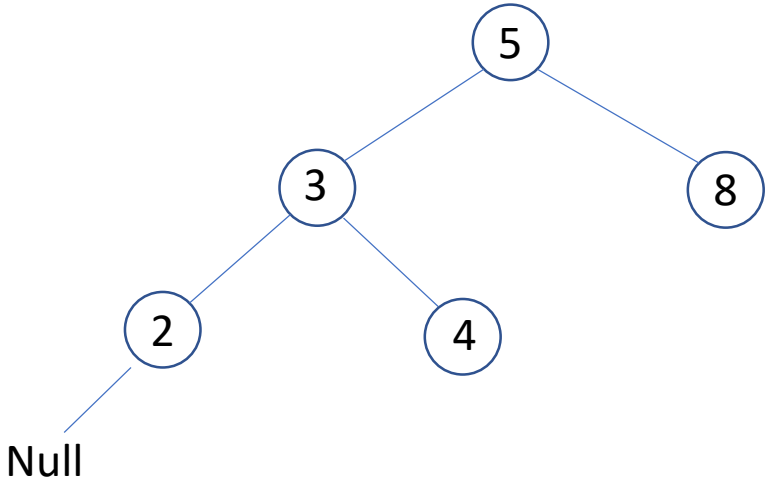
- Caminhamento central e impressão da árvore:




```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

Caminhamento Central

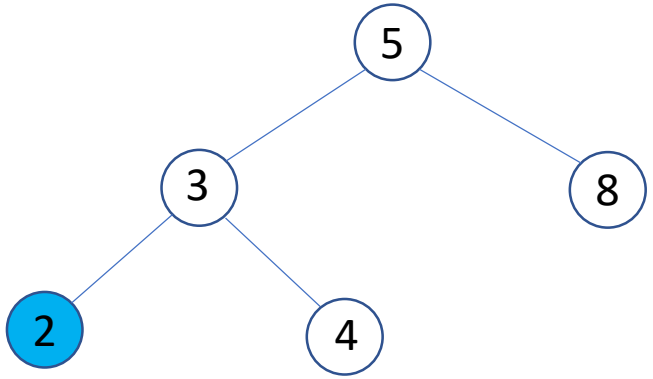
- Caminhamento central e impressão da árvore:




```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

Caminhamento Central

- Caminhamento central e impressão da árvore:

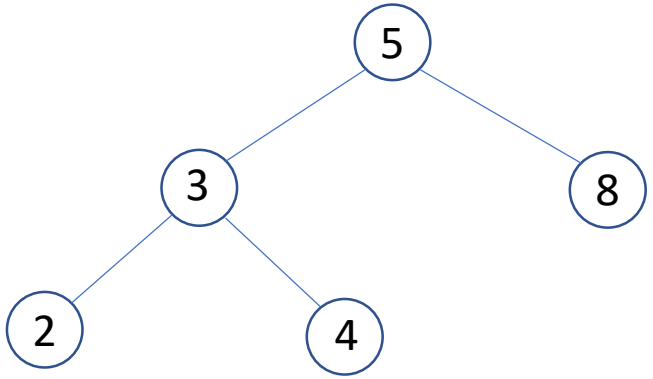



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2

Caminhamento Central

- Caminhamento central e impressão da árvore:

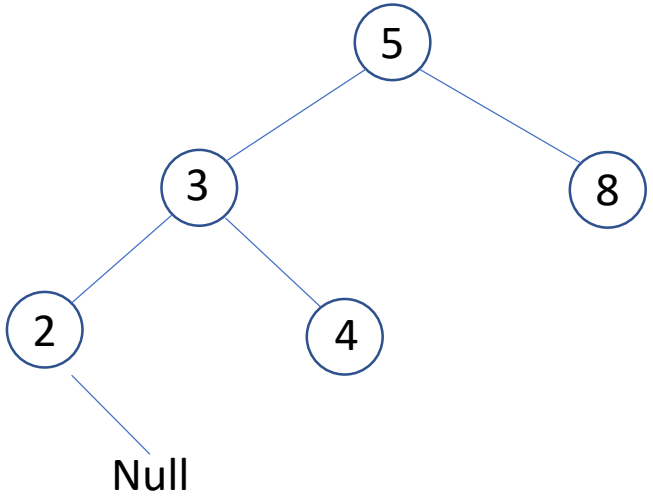



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2

Caminhamento Central

- Caminhamento central e impressão da árvore:

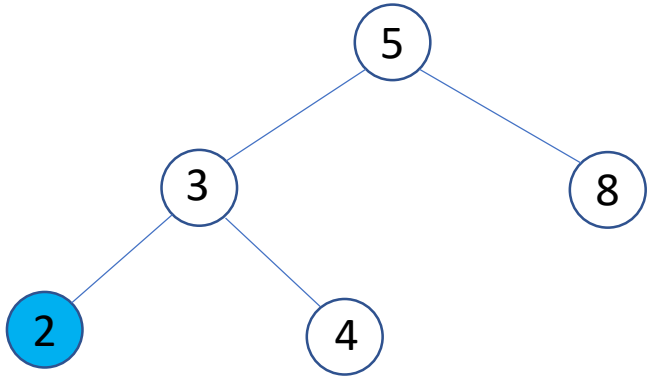



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2

Caminhamento Central

- Caminhamento central e impressão da árvore:

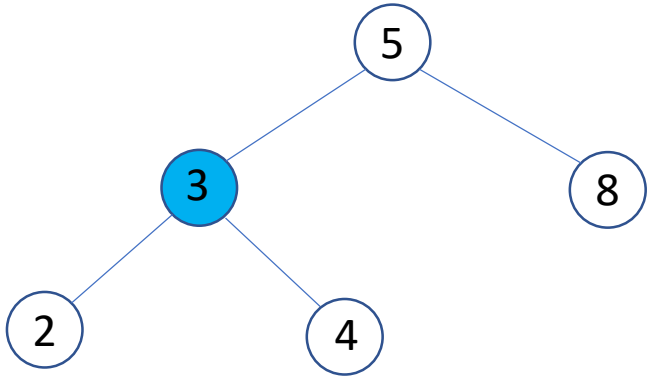



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2

Caminhamento Central

- Caminhamento central e impressão da árvore:

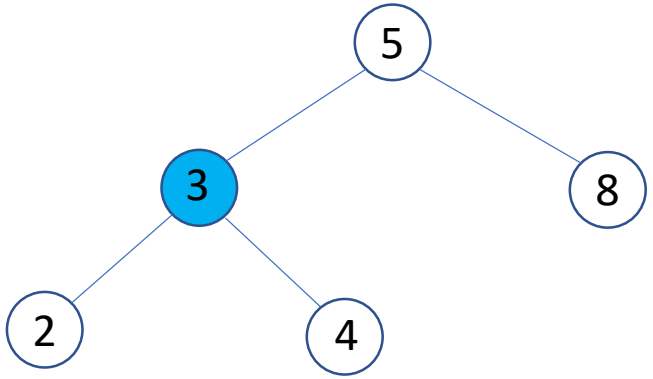



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2 3

Caminhamento Central

- Caminhamento central e impressão da árvore:

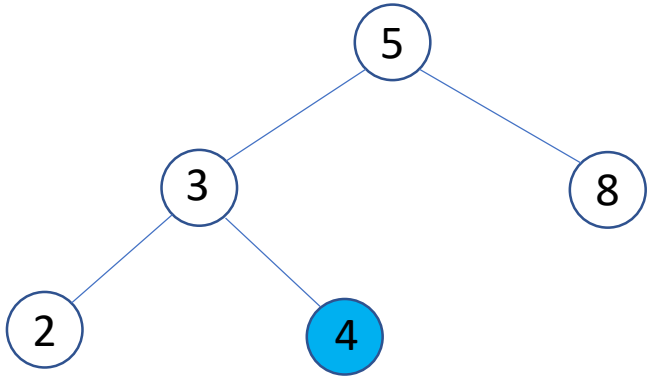



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2 3

Caminhamento Central

- Caminhamento central e impressão da árvore:

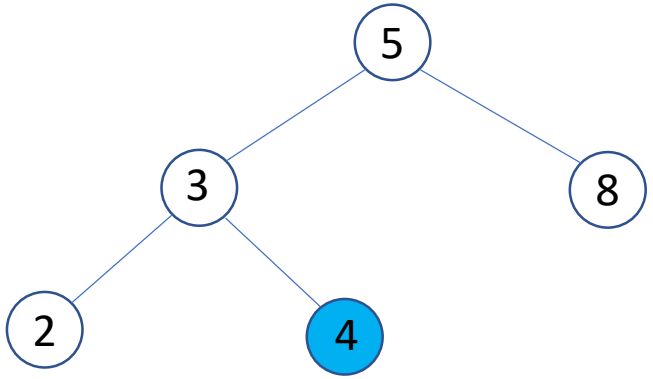



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2 3

Caminhamento Central

- Caminhamento central e impressão da árvore:

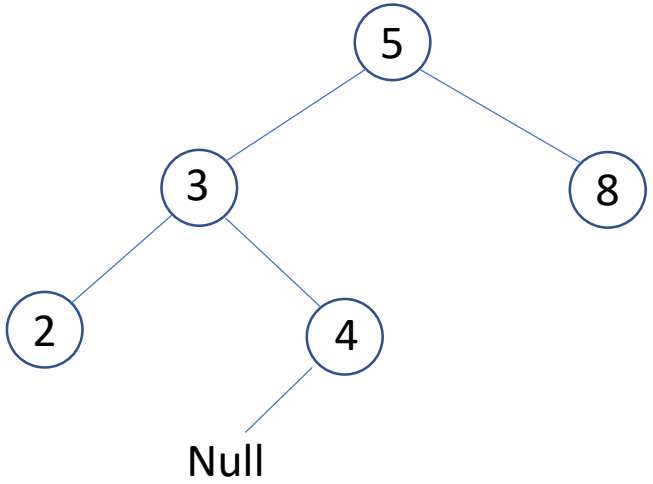



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2 3

Caminhamento Central

- Caminhamento central e impressão da árvore:

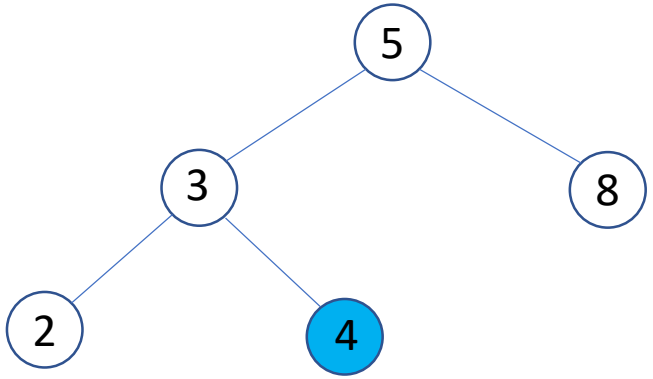



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2 3

Caminhamento Central

- Caminhamento central e impressão da árvore:

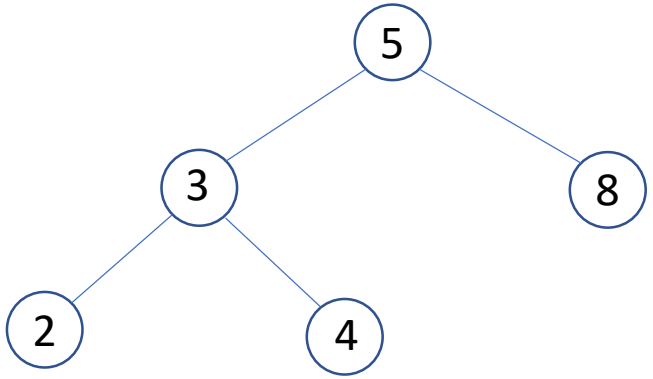



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2 3 4

Caminhamento Central

- Caminhamento central e impressão da árvore:

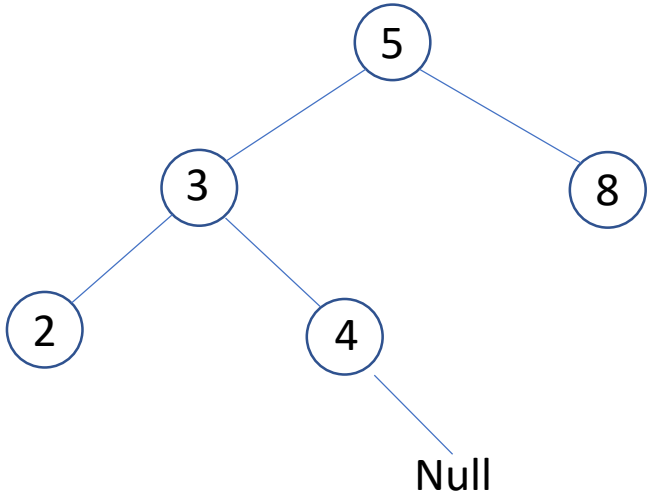



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2 3 4

Caminhamento Central

- Caminhamento central e impressão da árvore:

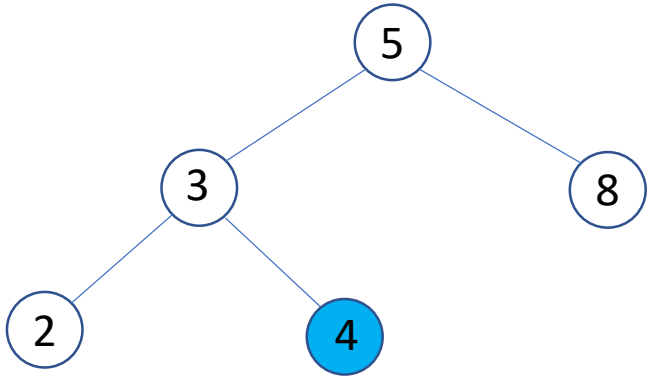



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2 3 4

Caminhamento Central

- Caminhamento central e impressão da árvore:

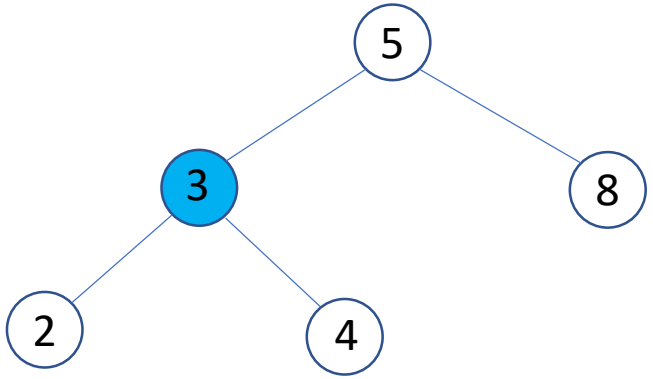



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2 3 4

Caminhamento Central

- Caminhamento central e impressão da árvore:

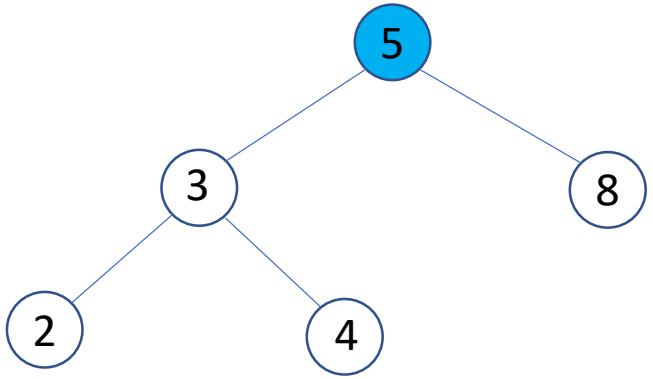



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2 3 4

Caminhamento Central

- Caminhamento central e impressão da árvore:

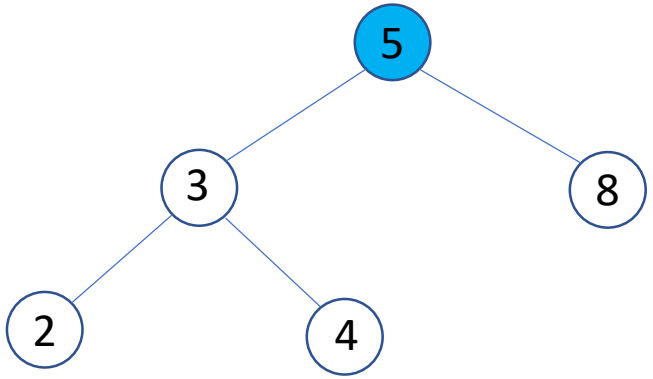



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2 3 4 5

Caminhamento Central

- Caminhamento central e impressão da árvore:

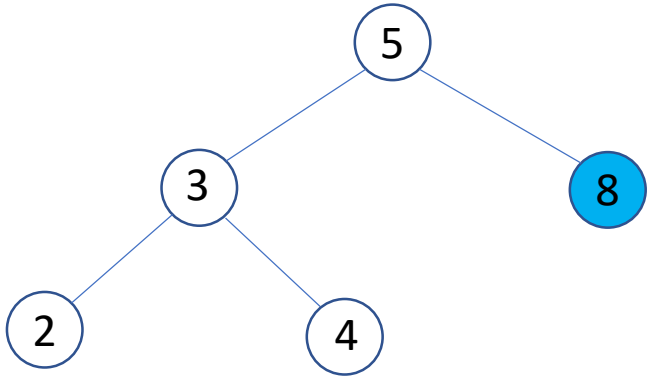



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2 3 4 5

Caminhamento Central

- Caminhamento central e impressão da árvore:

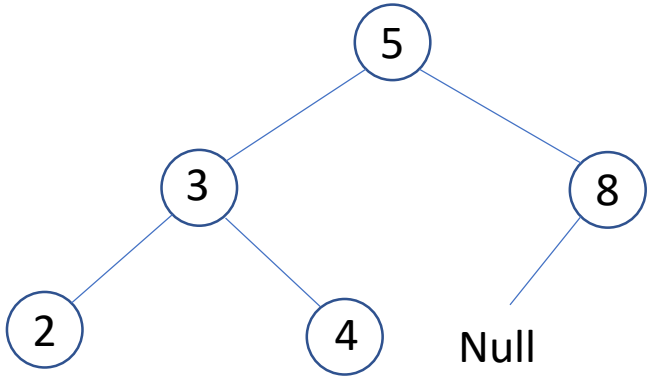



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2 3 4 5

Caminhamento Central

- Caminhamento central e impressão da árvore:

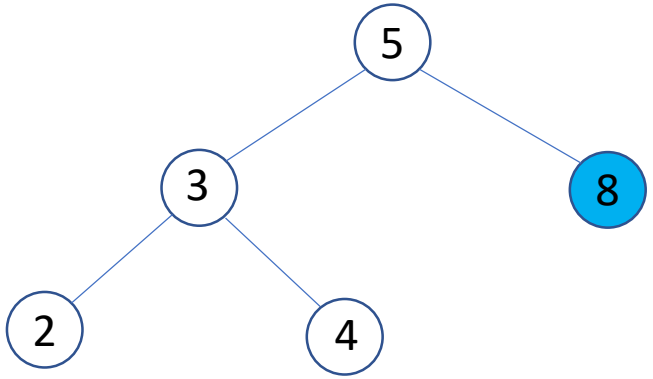



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2 3 4 5

Caminhamento Central

- Caminhamento central e impressão da árvore:

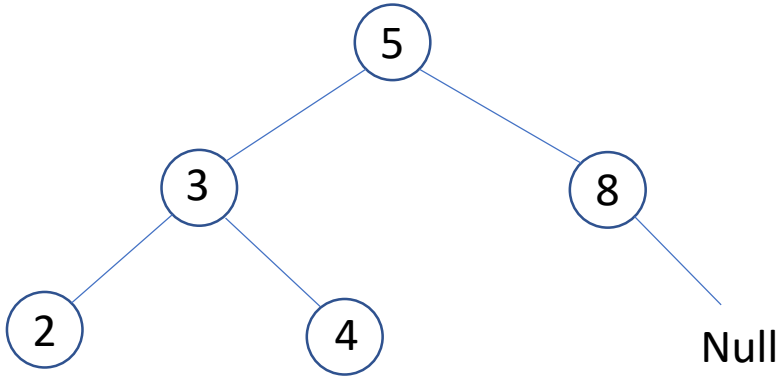



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2 3 4 5 8

Caminhamento Central

- Caminhamento central e impressão da árvore:

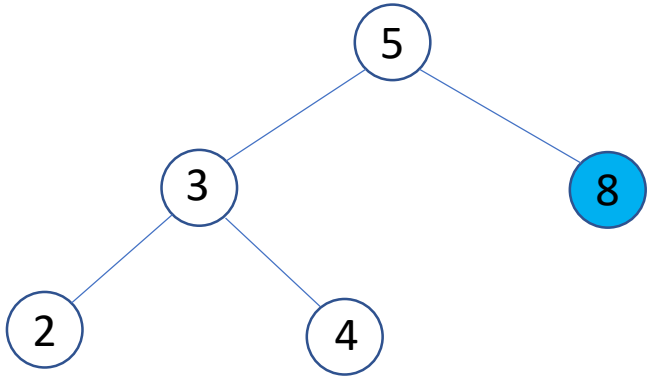



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2 3 4 5 8

Caminhamento Central

- Caminhamento central e impressão da árvore:

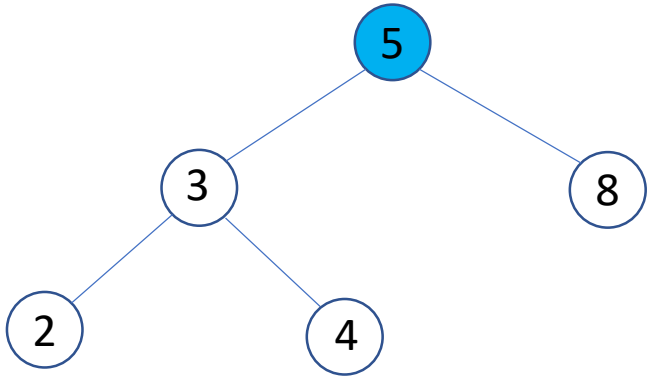



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2 3 4 5 8

Caminhamento Central

- Caminhamento central e impressão da árvore:

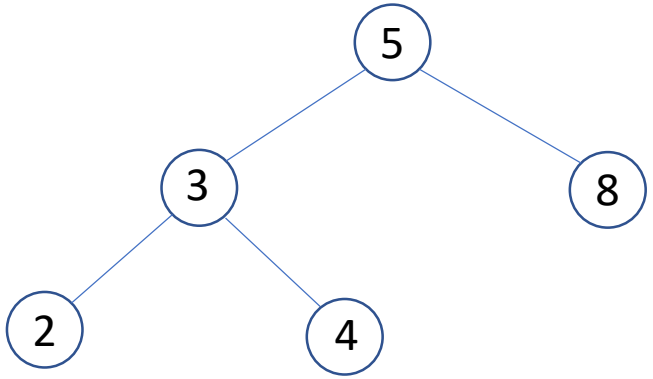



```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

2 3 4 5 8

Caminhamento Central

- Caminhamento central e impressão da árvore:




```
void Central(TipoApontador p) {
    if (p == NULL)
        return;
    Central(p->Esq);
    printf("%ld\n", p->Reg.Chave);
    Central(p->Dir);
}
```

Uma característica importante do caminhamento central é que os nós são visitados de **forma ordenada**.

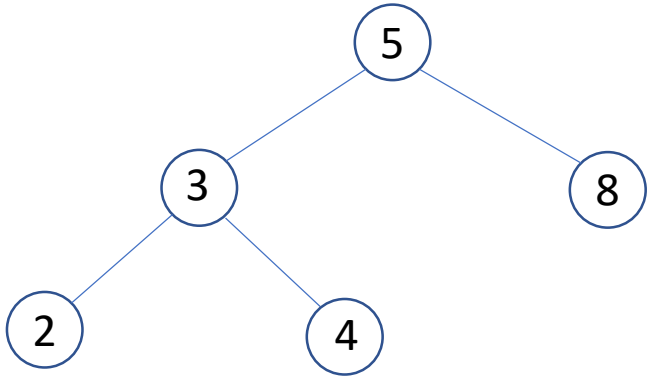
2 3 4 5 8

Caminhamento Pré-ordem

- O caminhamento em **pré-ordem** lista o **nó raiz da sub árvore**, **o nó filho da sub árvore à esquerda** e lista o **nó filho à direita**. (em cada subárvore lista os nós sempre da esquerda para a direita). O procedimento é aplicado recursivamente a toda árvore.
  - Em resumo: os nós de cada sub árvore são listados nesta ordem: **raiz, nós filhos à esquerda** e **nós filhos à direita**.

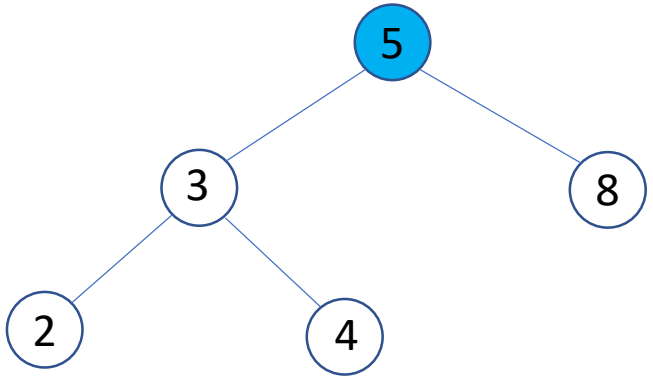
Caminhamento Pré-ordem

- Caminhamento Pré-ordem e impressão da árvore:



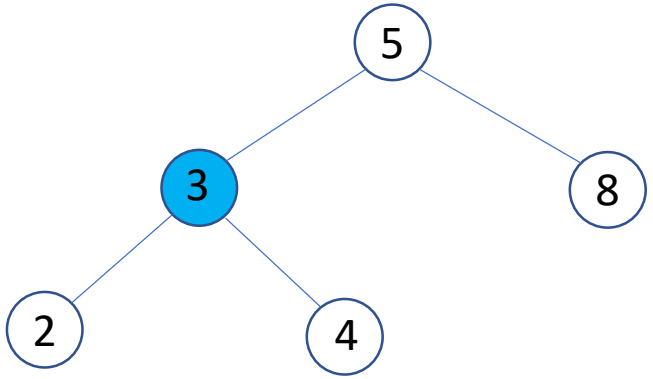
Caminhamento Pré-ordem

- Caminhamento Pré-ordem e impressão da árvore:



Caminhamento Pré-ordem

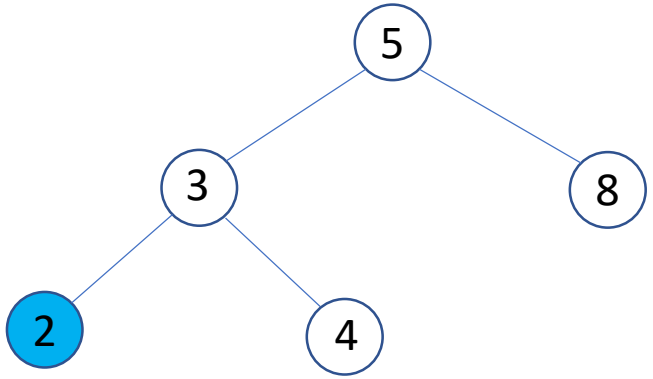
- Caminhamento Pré-ordem e impressão da árvore:



5 3

Caminhamento Pré-ordem

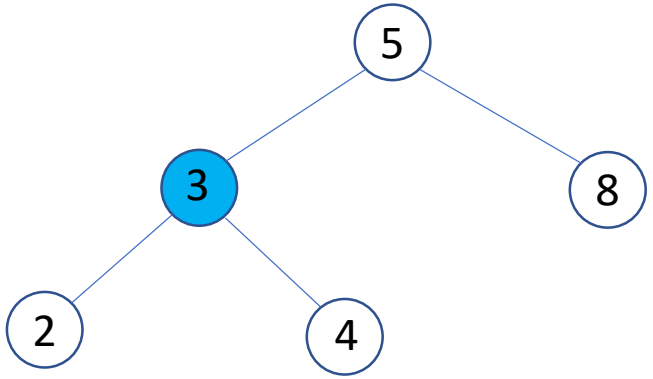
- Caminhamento Pré-ordem e impressão da árvore:



5 3 2

Caminhamento Pré-ordem

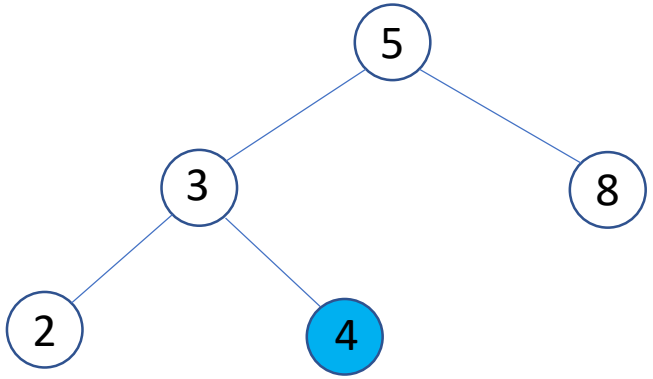
- Caminhamento Pré-ordem e impressão da árvore:



5 3 2

Caminhamento Pré-ordem

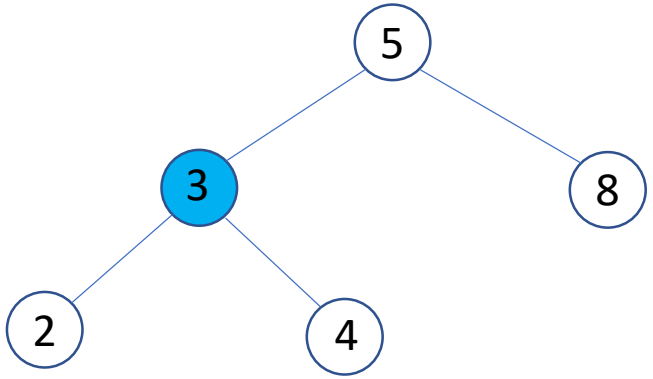
- Caminhamento Pré-ordem e impressão da árvore:



5 3 2 4

Caminhamento Pré-ordem

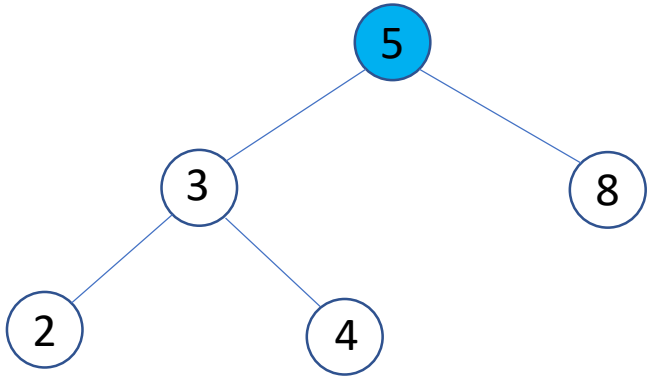
- Caminhamento Pré-ordem e impressão da árvore:



5 3 2 4

Caminhamento Pré-ordem

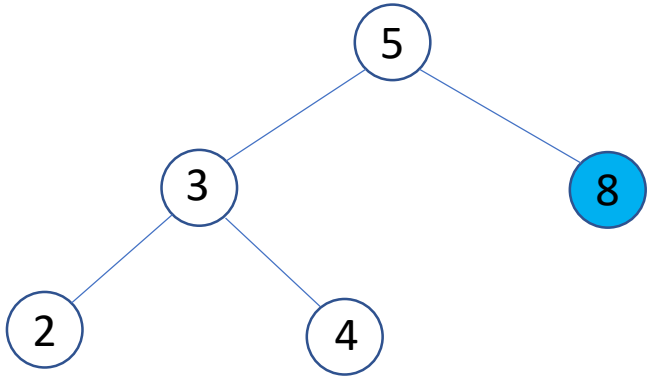
- Caminhamento Pré-ordem e impressão da árvore:



5 3 2 4

Caminhamento Pré-ordem

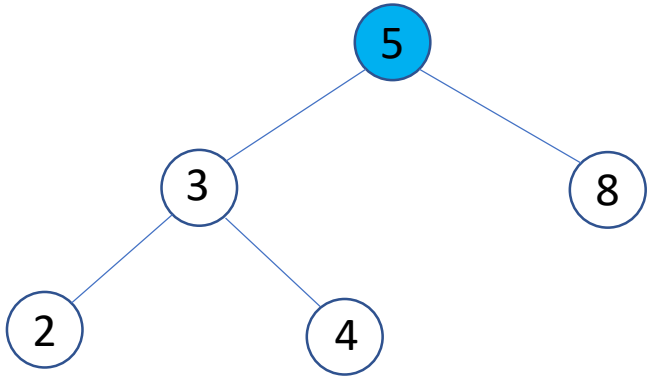
- Caminhamento Pré-ordem e impressão da árvore:



5 3 2 4 8

Caminhamento Pré-ordem

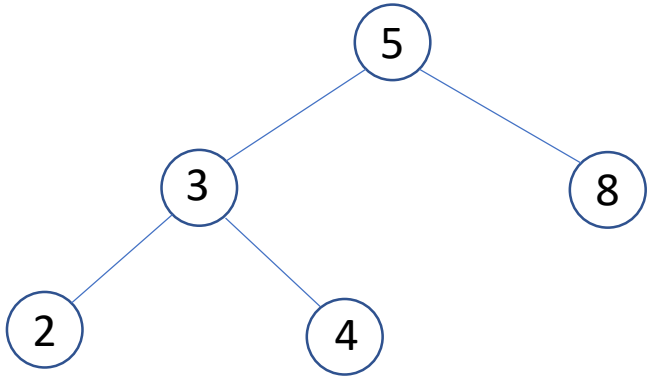
- Caminhamento Pré-ordem e impressão da árvore:



5 3 2 4 8

Caminhamento Pré-ordem

- Caminhamento Pré-ordem e impressão da árvore:



Observe que o nó raiz de cada sub árvore é listado primeiro.

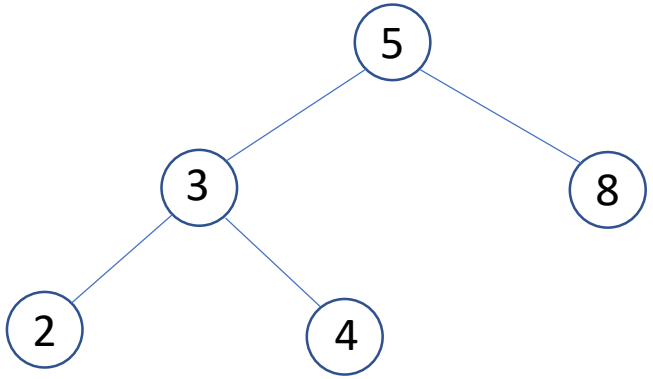
5 3 2 4 8

Caminhamento Pós-ordem

- O caminhamento em **pós-ordem** lista os **nós filhos à esquerda** da sub árvore, os **nós filhos da sub árvore à direita** e lista o **nó raiz** da sub árvore. (em cada sub árvore lista os nós sempre da esquerda para a direita). O procedimento é aplicado recursivamente a toda árvore.
  - Em resumo: os nós de cada sub árvore são listados nesta ordem: **nós filhos à esquerda**, **nós filhos à direita** e **raiz**.

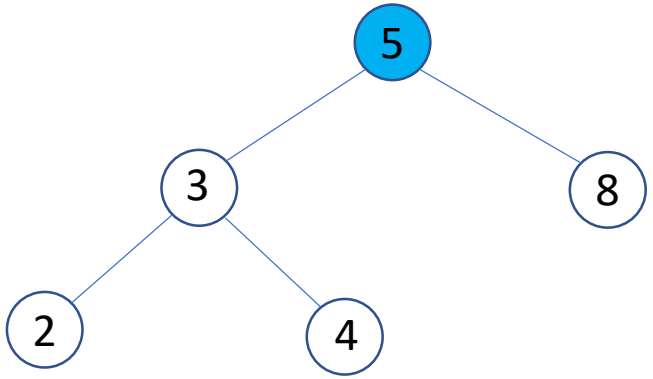
Caminhamento Pós-ordem

- Caminhamento Pós-ordem e impressão da árvore:



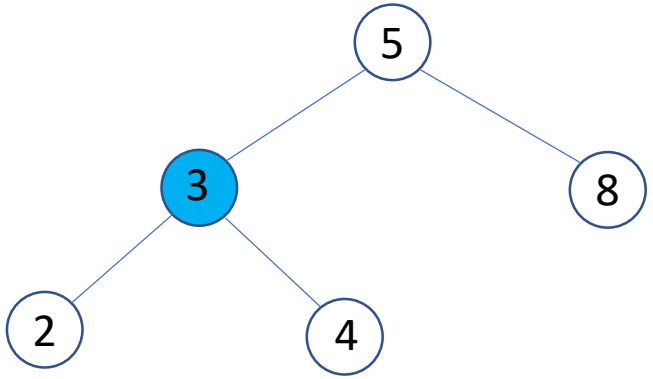
Caminhamento Pós-ordem

- Caminhamento Pós-ordem e impressão da árvore:



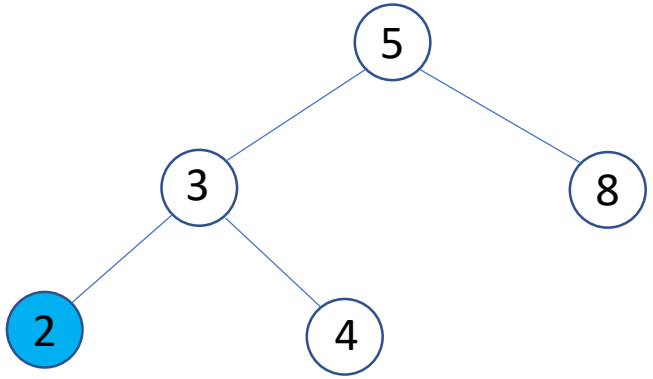
Caminhamento Pós-ordem

- Caminhamento Pós-ordem e impressão da árvore:



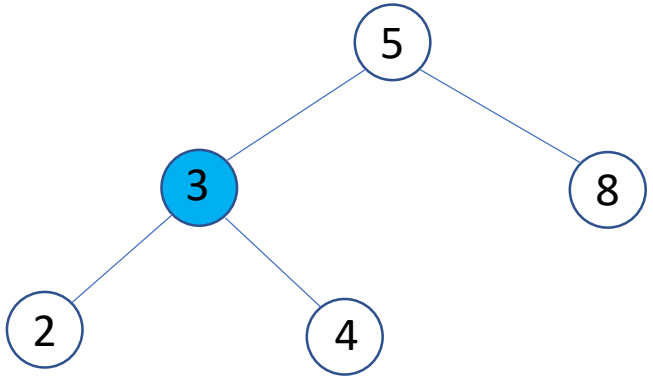
Caminhamento Pós-ordem

- Caminhamento Pós-ordem e impressão da árvore:



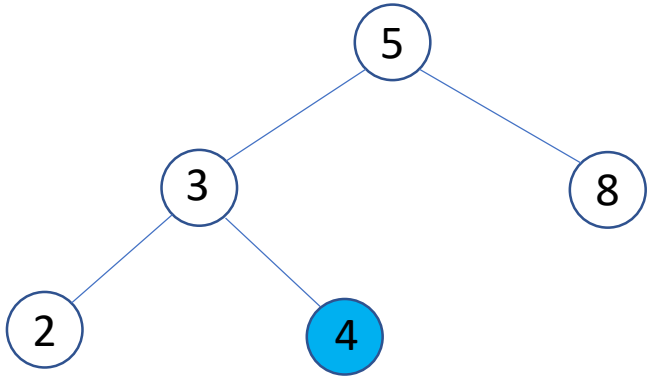
Caminhamento Pós-ordem

- Caminhamento Pós-ordem e impressão da árvore:



Caminhamento Pós-ordem

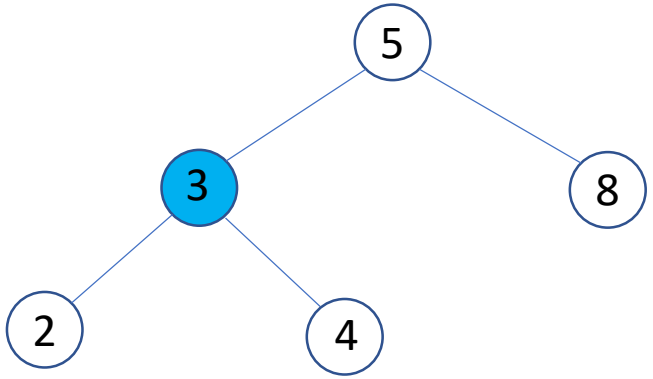
- Caminhamento Pós-ordem e impressão da árvore:



2 4

Caminhamento Pós-ordem

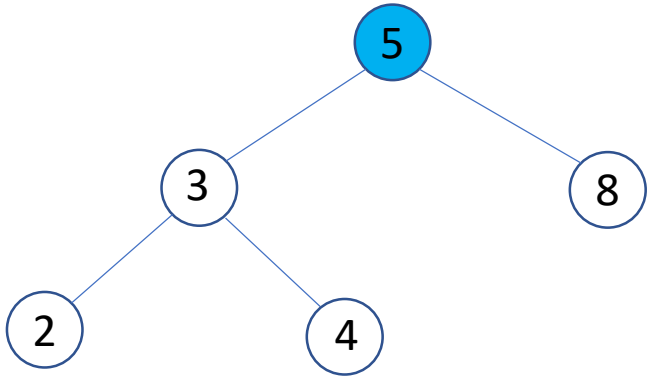
- Caminhamento Pós-ordem e impressão da árvore:



2 4 3

Caminhamento Pós-ordem

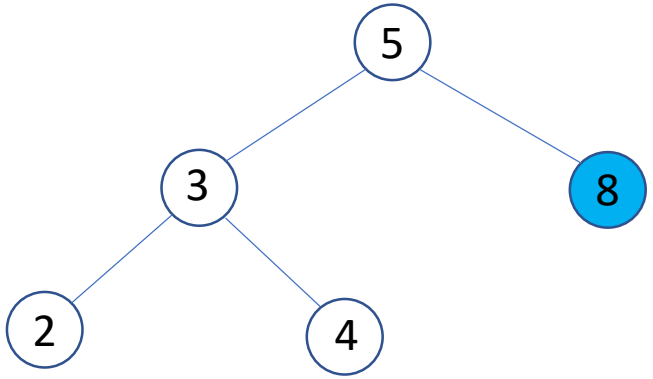
- Caminhamento Pós-ordem e impressão da árvore:



2 4 3

Caminhamento Pós-ordem

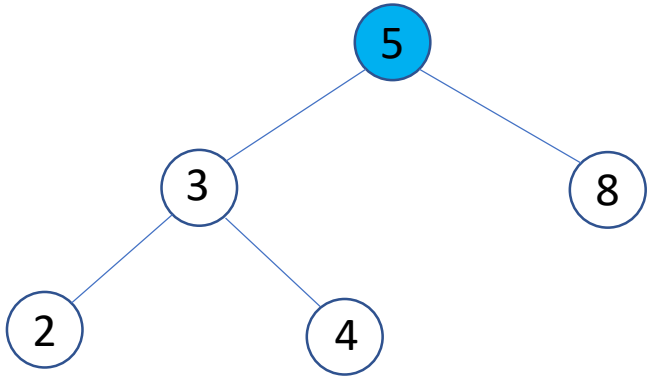
- Caminhamento Pós-ordem e impressão da árvore:



2 4 3 8

Caminhamento Pós-ordem

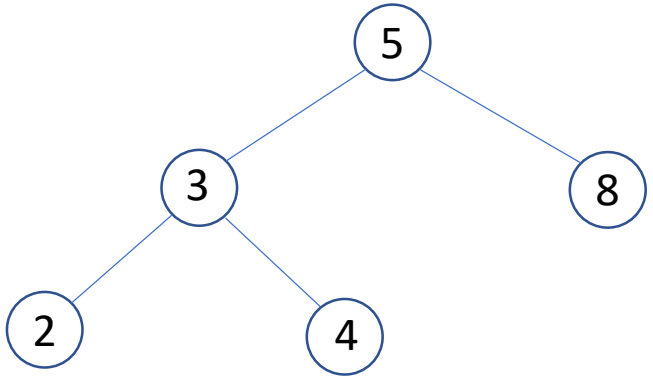
- Caminhamento Pós-ordem e impressão da árvore:



2 4 3 8 5

Caminhamento Pós-ordem

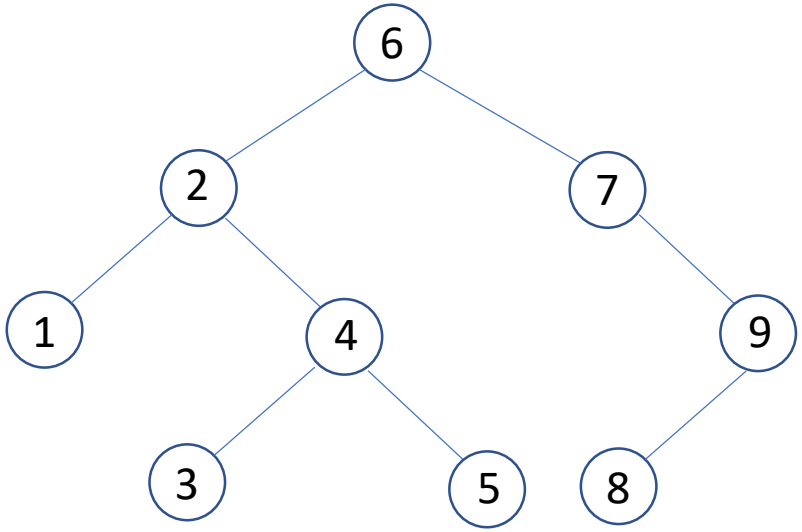
- Caminhamento Pós-ordem e impressão da árvore:



Observe que o nó raiz de cada sub árvore é listado por último.

2 4 3 8 5

Resumo sobre caminhamento



Pré-ordem: 6 – 2 – 1 – 4 – 3 – 5 – 7 – 9 – 8 (para cada sub árvore, a raiz é listada primeiro: **pré**)
Central: 1 – 2 – 3 – 4 – 5 – 6 – 7 – 8 – 9 (para cada sub árvore, a raiz é listada no centro: **central**)
Pós-Ordem: 1 – 3 – 5 – 4 – 2 – 8 – 9 – 7 – 6 (para cada sub árvore, a raiz é listada por último: **pós**)

Análise

- O número de comparações em uma pesquisa com sucesso:
  - melhor caso: $C(n) = (1)$
  - pior caso: $C(n) = (n)$ ← ???
  - caso médio: $C(n) = (Log\ n)$.

Análise

- O número de comparações em uma pesquisa com sucesso:
  - melhor caso: $C(n) = (1)$
  - pior caso: $C(n) = (n)$ ← ???
  - caso médio: $C(n) = (Log\ n)$.

- O tempo de execução dos algoritmos para árvores binárias de pesquisa depende muito do **formato das árvores**.

Análise

- Para obter o pior caso basta que as chaves sejam inseridas em ordem crescente ou decrescente.
  - Árvore se torna uma lista linear.
  - tempo médio de comparações: $(n + 1)/2$.

Exercícios

- Implemente a classe ArvoreBinaria, apresentada em sala, e teste você deve criar e inserir, pesquisar e remover uma série de elementos de uma árvore de pesquisa binária.
- Você deve acrescentar métodos à classe Arvore que permitam imprimir todas as informações dos elementos ou apenas as suas chaves durante a realização de caminhamentos (central, pré-ordem e pós-ordem).

Exercícios

- Modifique sua implementação de modo a contabilizar para sua classe árvore o número de comparações realizadas durante as operações de inserção, de pesquisa e de remoção.

Referências

Material adaptado do livro:

ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.

Os código presentes nos slides estão disponíveis no site do autor (link no último slide)

Referências

- Baseado no material de aulas de:
  - ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.
  - Para downloads dos códigos em Java:
  - http://www2.dcc.ufmg.br/livros/algoritmos-java/implementacoes-03.php

- Outras referências:
  - CORMEN, Thomas H. et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002. 916p. ISBN 8535209263 (Disponível no Acervo).

  - DROZDEK, Adam. Estrutura de dados e algoritmos em C++. São Paulo: Pioneira Thomson Learning, 2002. xviii, 579p. ISBN 8522102953 (Disponível no Acervo).